Machine Learning Techniques

Unit 2

Ankita Singh, Dept. of IT, AKGEC

Syllabus

- REGRESSION: Linear Regression and Logistic Regression
- BAYESIAN LEARNING - Bayes theorem, Concept learning, Bayes Optimal Classifier, Naïve Bayes classifier, Bayesian belief networks, EM algorithm.
- SUPPORT VECTOR MACHINE: Introduction, Types of support vector kernel – (Linear kernel, polynomial kernel,and Gaussiankernel), Hyperplane – (Decision surface), Properties of SVM, and Issues in SVM.

Regression

- In machine learning, regression is a *supervised learning task* aimed at predicting a continuous numerical value from input data.
- **It works by modeling the relationship between one or more independent variables (features/predictor) and a dependent variable (target/label).**
- The model is trained on labeled data, where both the inputs and corresponding outputs are known, and uses this learned relationship to make predictions on new, unseen data.
- More specifically, Regression analysis helps us to understand how the value of the dependent variable is changing corresponding to an independent variable when other independent variables are held fixed.

- **Example**: Imagine you want to predict the price of a house based on factors like its size, location, and number of rooms.
  - Each of these factors is called a **feature**, and the price is the **target**.
  - A **regression model** learns the relationship between these features and the target (house price) from existing data.
  - Once the model has learned this, it can be used to predict the price of a new house based on its features.

Terminologies Related to Regression Analysis

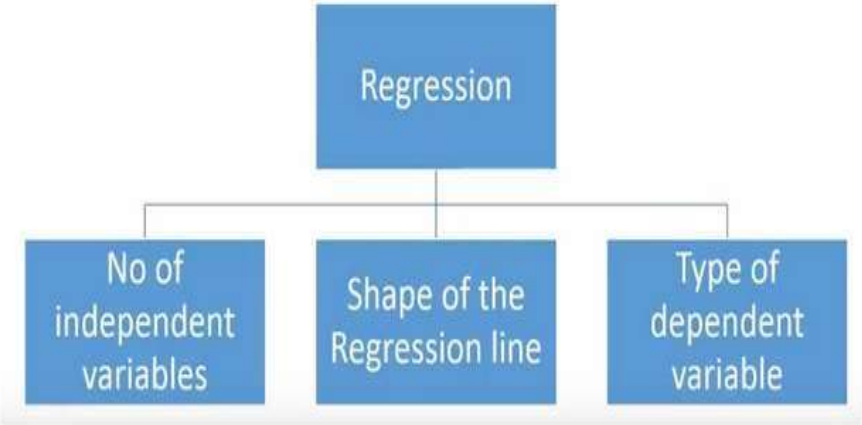
- **Dependent Variable:** The main factor in Regression analysis which we want to predict or understand is called the dependent variable. It is also called **target variable/label.**
- **Independent Variable:** The factors which affect the dependent variables or which are used to predict the values of the dependent variables are called independent variable, also called as a **predictor/feature**.
- **Outliers:** Outlier is an observation which contains either very low value or very high value in comparison to other observed values. An outlier may hamper the result, so it should be avoided.
- **Underfitting and Overfitting:** If our algorithm works well with the training dataset but not well with test dataset, then such problem is called **Over fitting**. And if our algorithm does not perform well even with training dataset, then such problem is called **under fitting**.

Regression is a common technique in machine learning and is used in many applications, including:

- Forecasting
- Predicting healthcare trends
- Forecasting house prices
- Predicting stock prices
- Predicting salary changes
- Predicting portfolio performance

Types of Regression

There are various kinds of **regression techniques available to make predictions**. These techniques are mostly **driven by three metrics** (number of independent variables, type of dependent variables and shape of regression line).



There are many types of Regression.
But right now we have to focus on only two types which are :

1. Linear Regression
2. Logistic Regression

Linear Regression

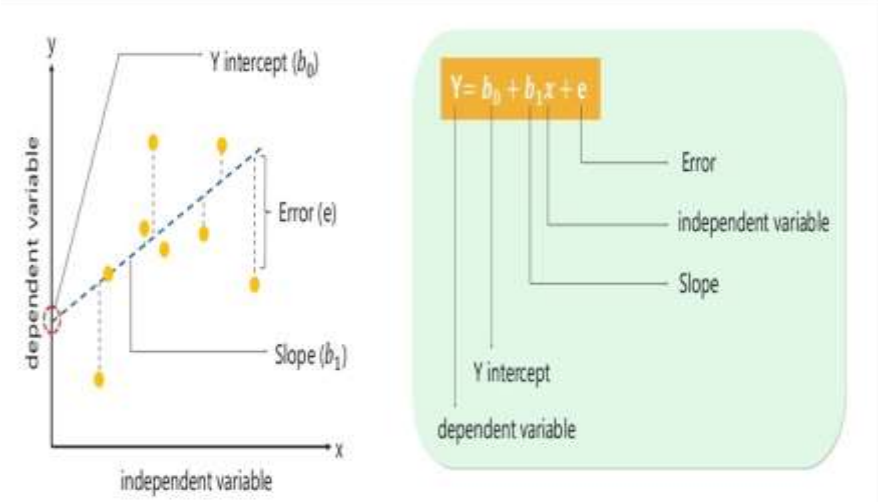
- Linear regression is used when the **relationship between the dependent and independent variables is assumed to be linear**. It is suitable for continuous numerical data and when a straight line can predict the response variable.
- Linear regression analysis is used to **predict the value of a variable based on the value of another variable or variables.** The variable you want to predict is called the **dependent variable(target/output).**The variable you are using to predict variable value is called the **independent variable**
- Which means it finds how the value of the **dependent variable is changing according to the value of the independent variable.**

Linear Regression Equation

Linear Regression establishes a linear relationship between dependent variable (Y) and one or more independent variables (X) using a **best fit straight line**(also known as regression line).

- The **slope (b_1)** tells how much Y changes when X changes.
- The **Y-intercept (b_0)** is where the line starts on the Y-axis.
- **Dots** are actual data points, and the **errors (e)** show how far these points are from the predicted line.

The aim of linear regression is to find the regression coefficients that produce the best-fitted line.

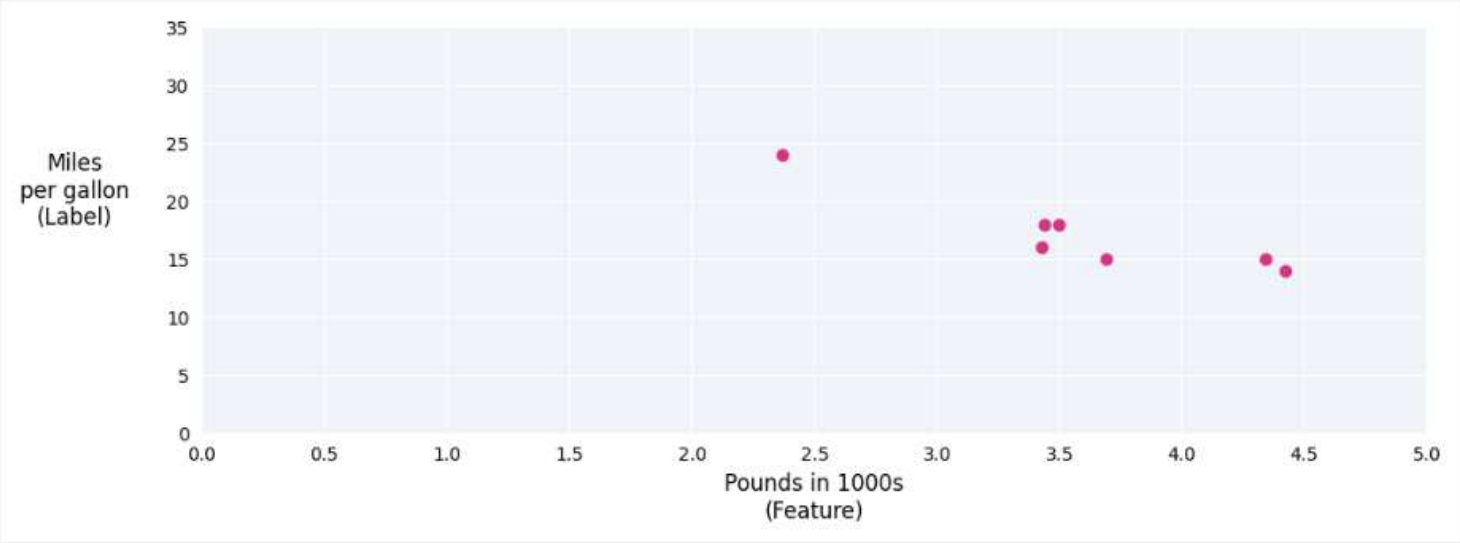


Example

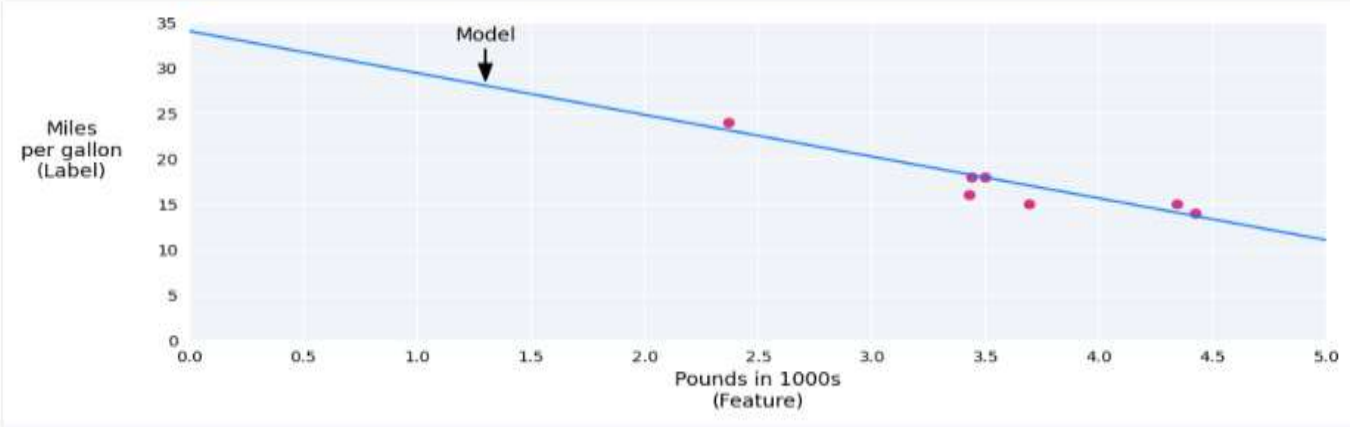
For example, suppose we want to predict a car's fuel efficiency in miles per gallon based on how heavy the car is, and we have the following dataset:

Pounds in 1000s (feature)	Miles per gallon (label)
3.5	18
3.69	15
3.44	18
3.43	16
4.34	15
4.42	14
2.37	24

If we plotted these points, we'd get the following graph:



We could create our own model by drawing a best fit line through the points:



In ML, we write the equation for a linear regression model as follows:

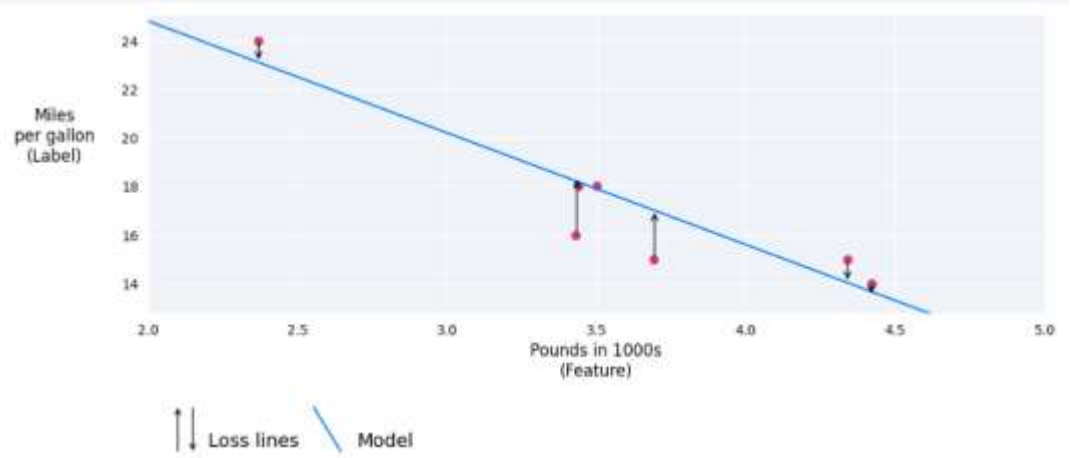
$$y' = b + w_1 x_1$$

Prediction Bias Weight Feature value

Calculated from training

How to obtain best fit line

- This task can be easily accomplished by **Least Square Method**.
- It is the most common method used for fitting a regression line.
- It calculates the **best-fit line** for the observed data by minimizing the **sum of the squares of the vertical deviations** from each data point to the line.



L_2 loss	$\sum(\text{actual value} - \text{predicted value})^2$
Mean squared error (MSE)	$\frac{1}{N}\sum(\text{actual value} - \text{predicted value})^2$

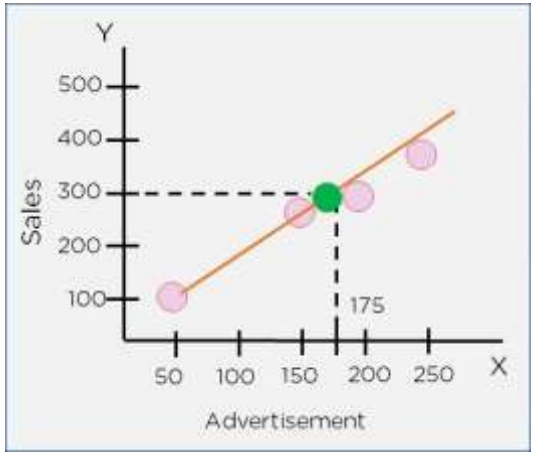
Use case of Linear Regression

1. Using Linear Regression, we can plot the graph of Sales vs. Advertising, and find the line of best fit between them, and, using that, find the values of the missing variable.

2. Using regression, given the advertisement amount, you can predict how many sales will take place

3. Here we have the answer:

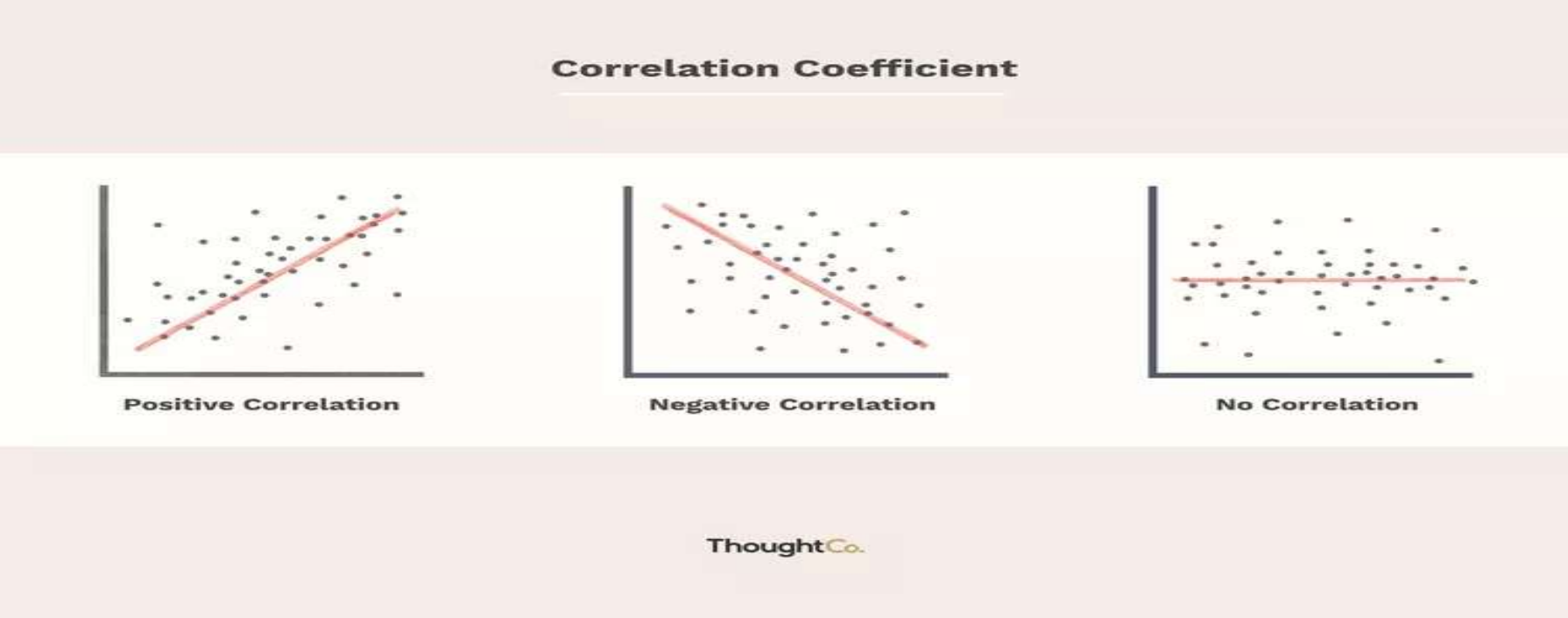
Advertisement	Sales
50	100
150	275
200	300
250	400
175	??



Advertisement	Sales
50	100
150	275
200	300
250	400
175	300

Correlation coefficient

- The correlation coefficient is a statistical measure of the strength of a linear relationship between two variables. Its values can **range from -1 to 1.**
- **Positive Correlation** -The value of one variable increases linearly with increase in another variable. This indicates a similar relation between both the variables. So its correlation coefficient would be positive or 1 in this case.
- **Negative Correlation**- When there is a decrease in values of one variable with increase in values of other variable. In that case, correlation coefficient would be negative.
- **Zero Correlation or No Correlation**- There is one more situation when there is no specific relation between two variables.


Correlation Coefficient
Positive Correlation
Negative Correlation
No Correlation
ThoughtCo.

- The linear correlation coefficient defines the degree of relation between two variables and is denoted by “r”. If x & y are the two variables of discussion, then the correlation coefficient can be calculated using the formula

Here,

$$r = \frac{n(\sum xy) - (\sum x)(\sum y)}{[n\sum x^2 - (\sum x)^2][n\sum y^2 - (\sum y)^2]}$$

n = Number of values or elements

∑x = Sum of 1st values list

∑y = Sum of 2nd values list

∑xy = Sum of the product of 1st and 2nd values

∑x2 = Sum of squares of 1st values

∑y2 = Sum of squares of 2nd values

- Regression coefficient of Y on X
- y=a+bx

$$b_{yx} = \frac{N\Sigma XY - (\Sigma X)(\Sigma Y)}{N\,\Sigma X^2 - (\Sigma X)^2}$$

Multiple Linear Regression

- The difference between simple linear regression and multiple linear regression is that **multiple linear regression has (>1) independent variables, whereas simple linear regression has only 1** independent variable.
- **Multiple linear equation be like…**

$$Y=a+b_1X_1+b_2X_2+\ldots\ldots\ldots\ldots b_nXn$$

Assignment

Fit a linear regression model for the dataset (x, y): (1, 1.5), (2, 3.0), (3,4.5), (4, 6.0) and predict y for x = 5.

- Advantages of Linear Regression:
- **Easy to Use**: It's simple to understand and set up.
- **Clear Results**: You can easily see how one thing affects another.
- **Quick Calculations**: It works fast, even with a lot of data.
- **Works Well for Straight Lines**: It’s good when the data forms a straight line.
- **Builds a Foundation**: It helps you learn about more advanced methods later.

Disadvantages of Linear Regression:

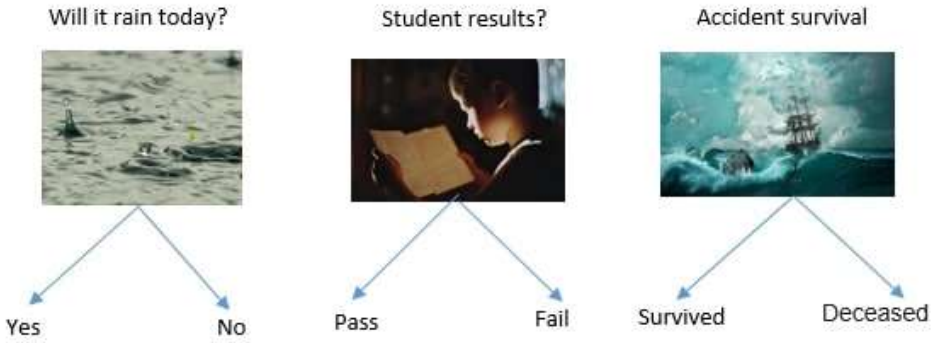
- **Only for Straight Lines**: It doesn’t work well if the relationship isn’t straight.
- **Easily Affected by Outliers**: Extreme values can change the results a lot.
- **Struggles with Complexity**: It can’t handle complicated relationships between data.
- **Needs Independent Data**: It assumes that the data points don’t influence each other.
- **Requires Consistent Variance**: It expects that the spread of errors is the same across the data, which isn’t always true.

Logistic Regression

- **Logistic regression** is designed to predict the probability of a given outcome.
- Many problems require a probability estimate as output. **Logistic regression** is an extremely efficient mechanism for calculating probabilities.
- Practically speaking, you can use the returned probability in either of the following two ways:
  a. Applied "as is." For example, if a spam-prediction model takes an email as input and outputs a value of 0.932, this implies a 93.2% probability that the email is spam.
  b. Converted to a **binary category** such as True or False, Spam or Not Spam.

Logistic Regression

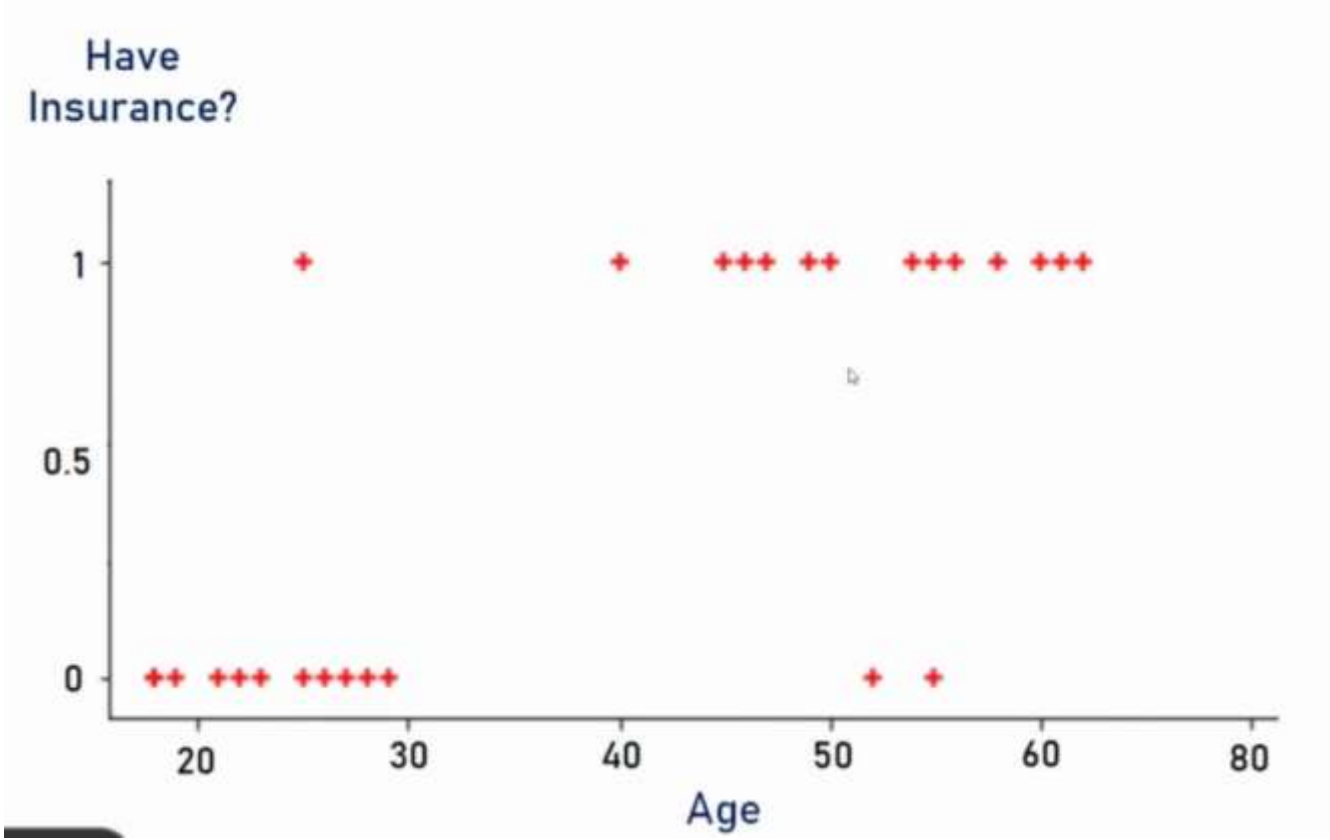
- **Logistic regression** is a type of machine learning model used for **classification** tasks, where the goal is to predict whether something belongs to one of two categories, like yes/no, true/false, or spam/not spam.
- Instead of predicting a continuous number like in linear regression, logistic regression predicts the **probability** that an input belongs to a particular class (like the probability that an email is spam). The result is a value between 0 and 1.

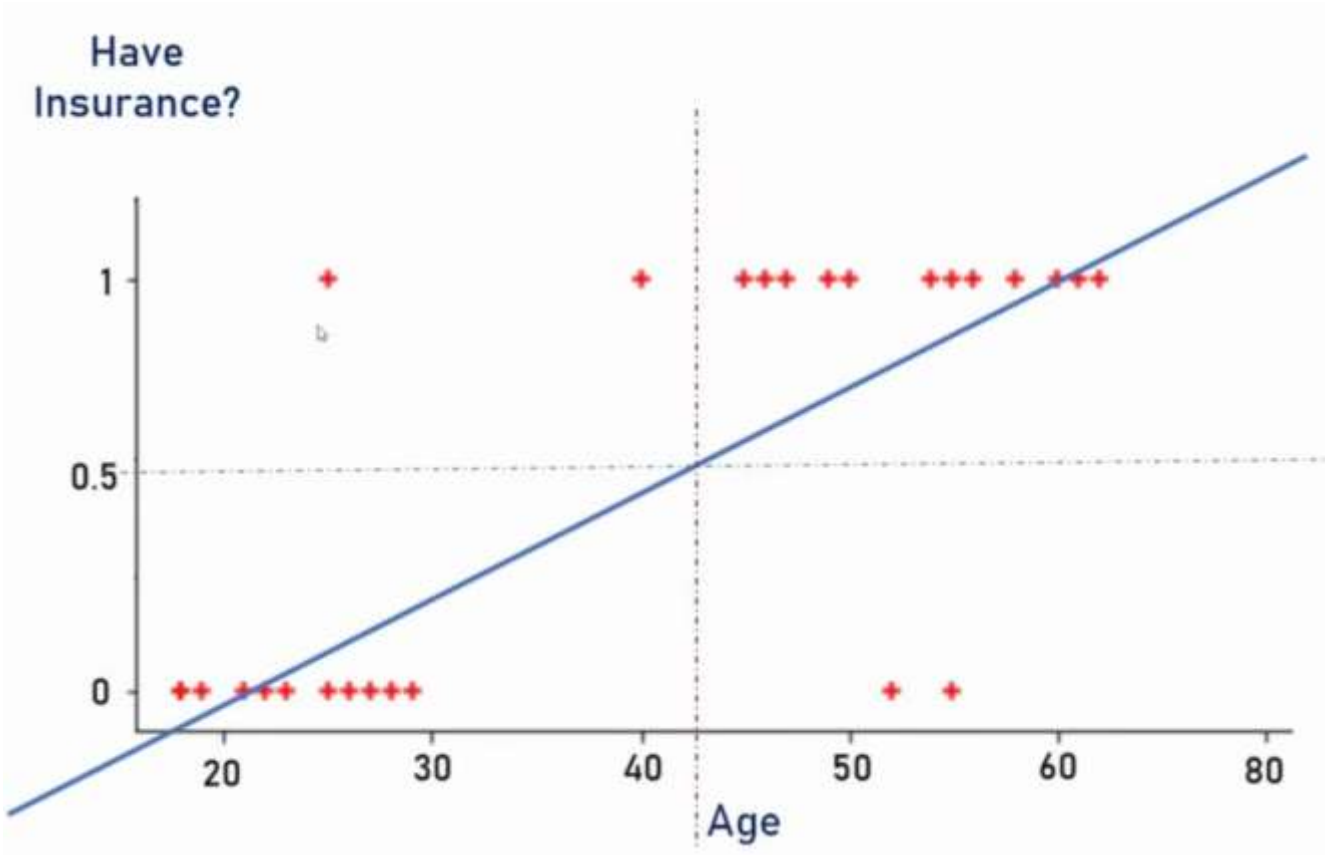


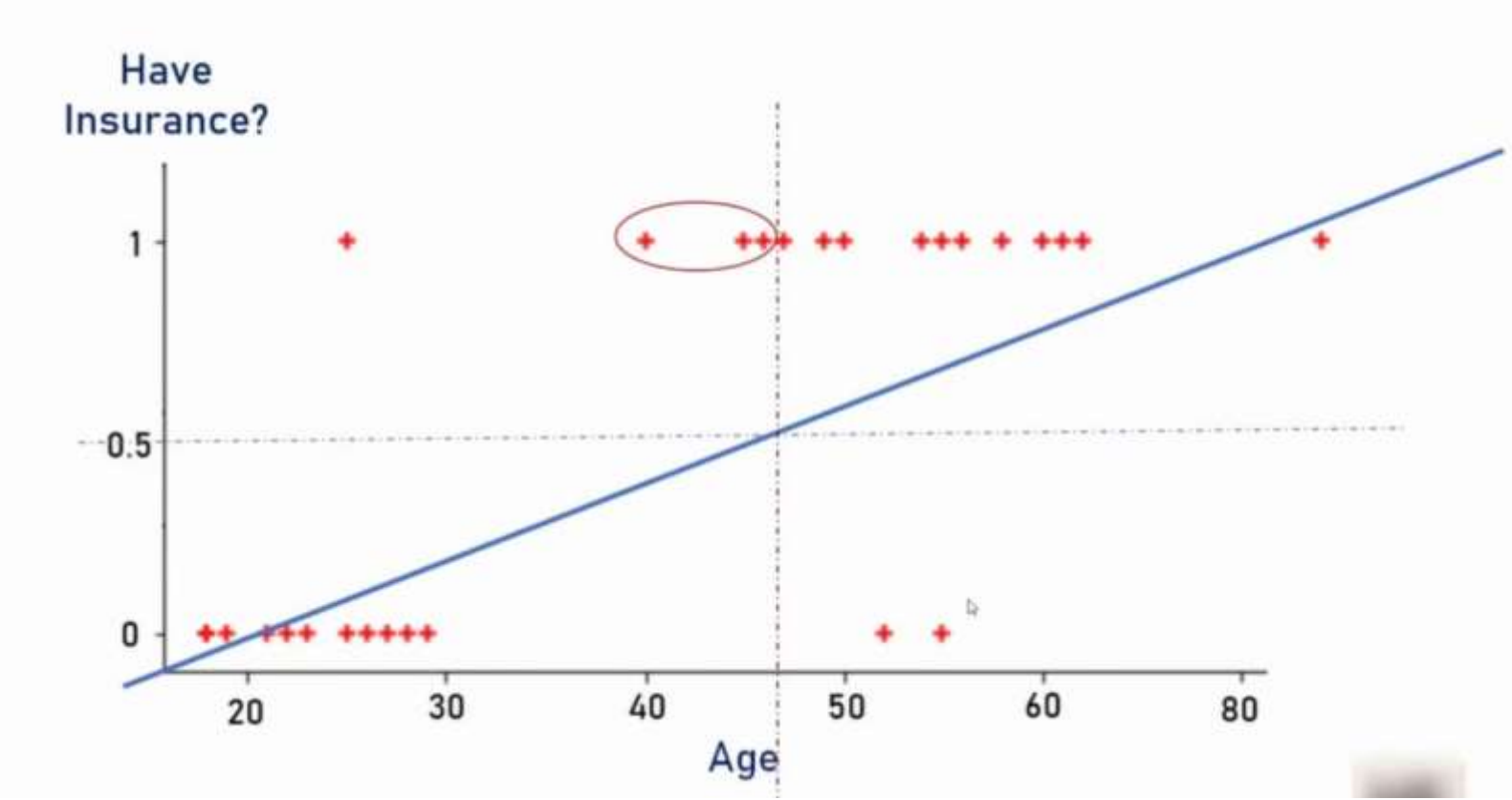
Example

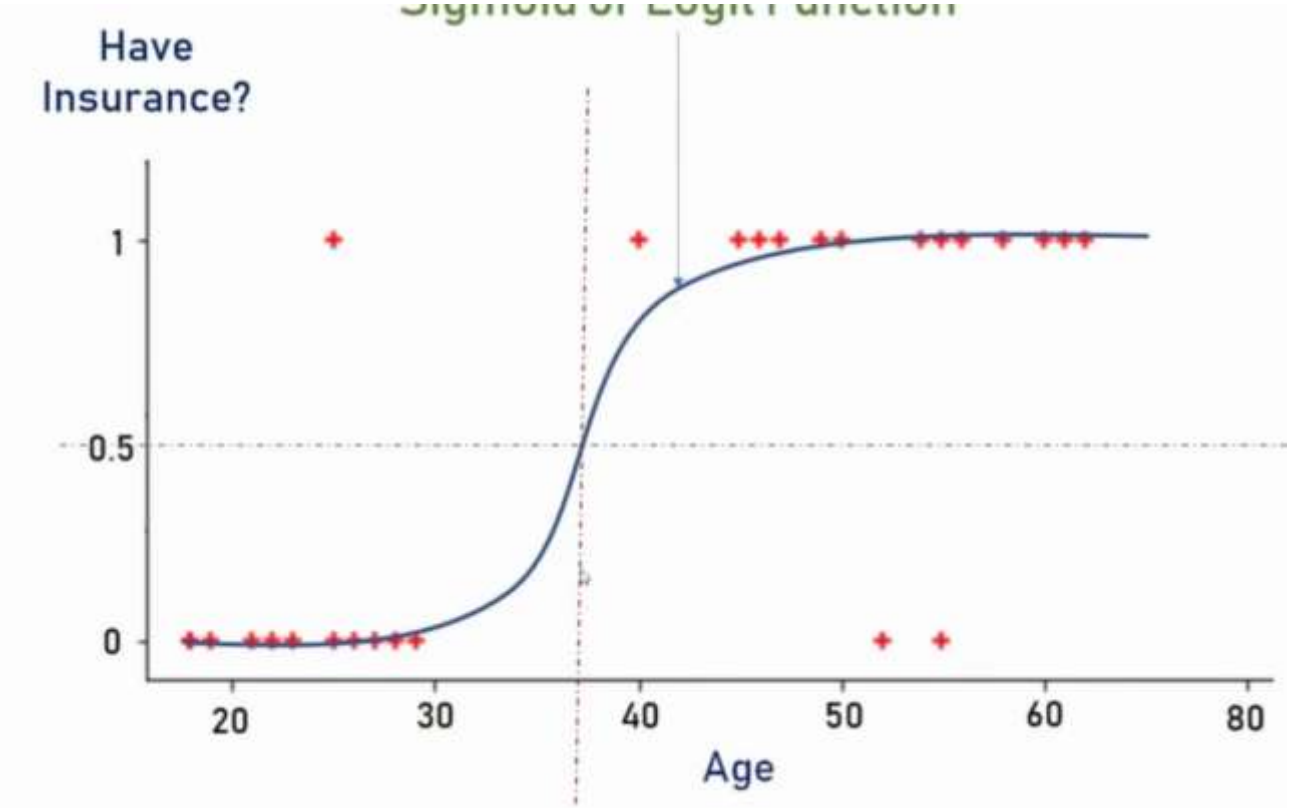
To predict if person will have insurance based on age.

age	have_insurance
22	0
25	0
47	1
52	0
46	1
56	1
55	0
60	1
62	1
61	1
18	0
28	0
27	0
29	0
49	1

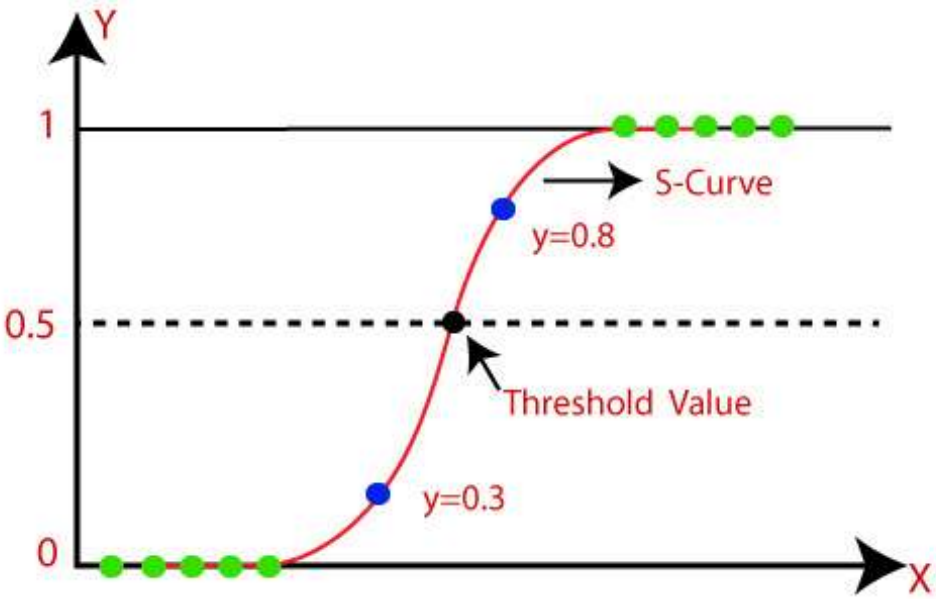

Have
Insurance?
1
0.5
0
20
30
40
50
60
80
Age


Have
Insurance?
1
0.5
0
20
30
40
50
60
80
Age


Have
Insurance?
1
0.5
0
20
30
40
50
60
80
Age

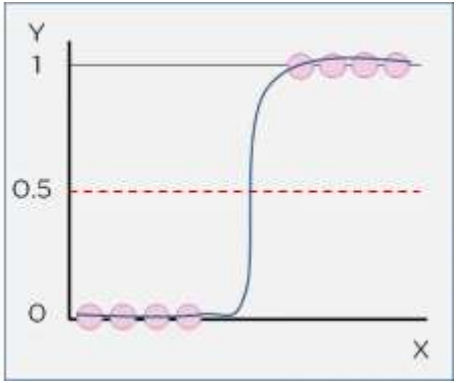

Have Insurance?
1
0.5
0
20
30
40
50
60
80
Age

- In Logistic regression, instead of fitting a regression line, we fit an "S" shaped logistic function, which predicts two maximum values (0 or 1).
- Logistic Regression can be used to classify the observations using different types of data and can easily determine the most effective variables used for the classification. The below image is showing the logistic function:



Logistic Function (Sigmoid Function)

- The sigmoid function is a mathematical function used to map the predicted values to probabilities.
- It maps any real value into another value within a range of 0 and 1.
- The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the "S" form. The S-form curve is called the Sigmoid function or the logistic function.
- In logistic regression, we use the concept of the threshold value, which defines the probability of either 0 or 1. Such as values above the threshold value tends to 1, and a value below the threshold values tends to 0.



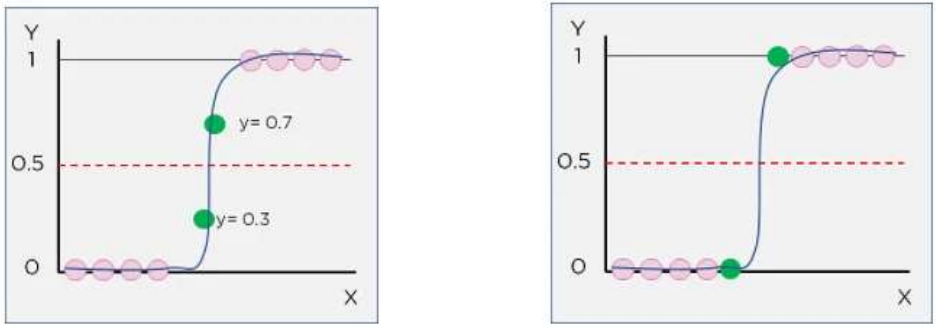


Figure 11: Dividing data into categories

Sigmoid Function

To ensure output of model represents a probability, always outputting a value between 0 and 1. we use standard logistic function, also known as the **sigmoid function** (*sigmoid* means "s-shaped"), has the formula:

$$f(x) = \frac{1}{1+e^{-x}}$$

Figure 1 shows the corresponding graph of the sigmoid function.

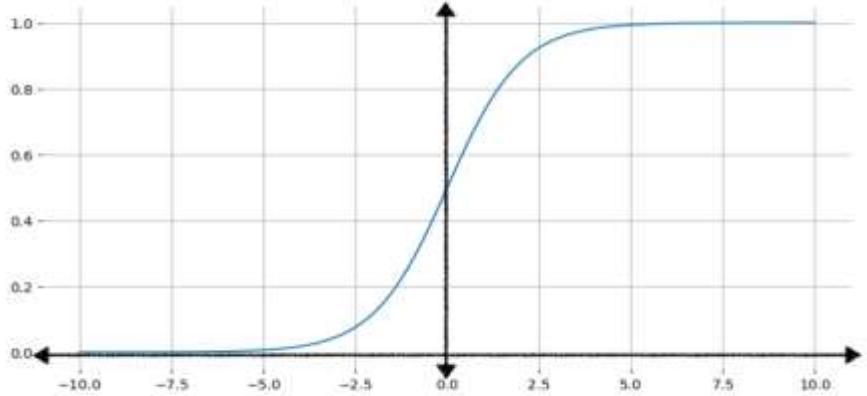


Figure 1. *Graph of the sigmoid function. The curve approaches 0 as x values decrease to negative infinity, and 1 as x values increase toward infinity.*

As the input, x, increases, the output of the sigmoid function output approaches but never 1.
Similarly as the input decreases, the sigmoid function's output approaches but never 0

Transforming linear output using the sigmoid function

The following equation represents the linear component of a logistic regression model:

$$z = b + w_1x_1 + w_2x_2 + \ldots + w_Nx_N$$

where:

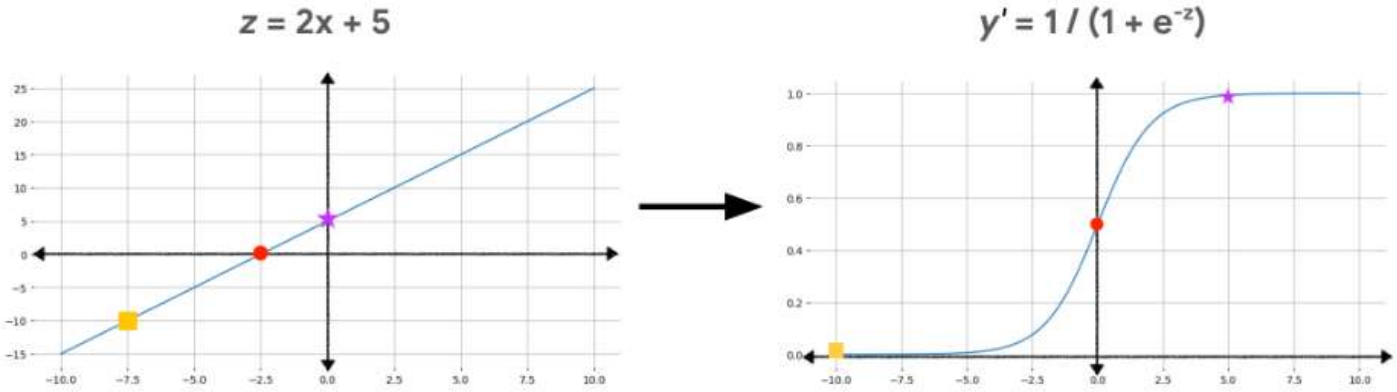
- z is the output of the linear equation, also called the **log odds**.
- b is the bias.
- The w values are the model's learned weights.
- The x values are the feature values for a particular example.

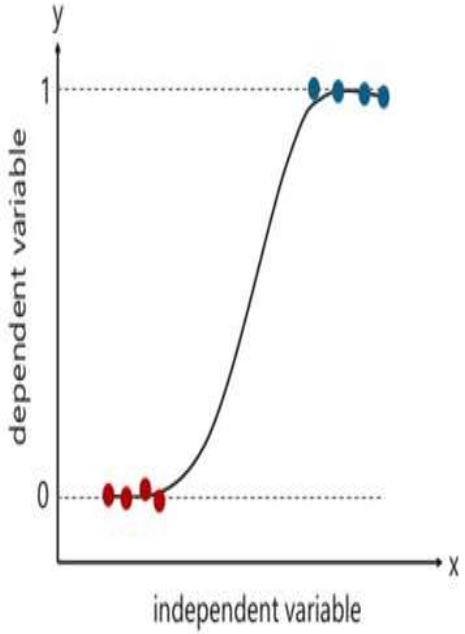
To obtain the logistic regression prediction, the z value is then passed to the sigmoid function, yielding a value (a probability) between 0 and 1:

$$y' = \frac{1}{1 + e^{-z}}$$

where:

- y' is the output of the logistic regression model.
- z is the linear output (as calculated in the preceding equation).


z = 2x + 5
y' = 1 / (1 + e^-z)



- *Dependent variable (Y):*
The response binary variable holding values like 0 or 1, Yes or No, A, B or C

- *Independent variable (X):*
The predictor variable used to predict the response variable.

The following equation is used to represent a linear regression model:

$$\log\left(\frac{Y}{1-Y}\right) = C + B1X1 + B2X2 + \ldots$$

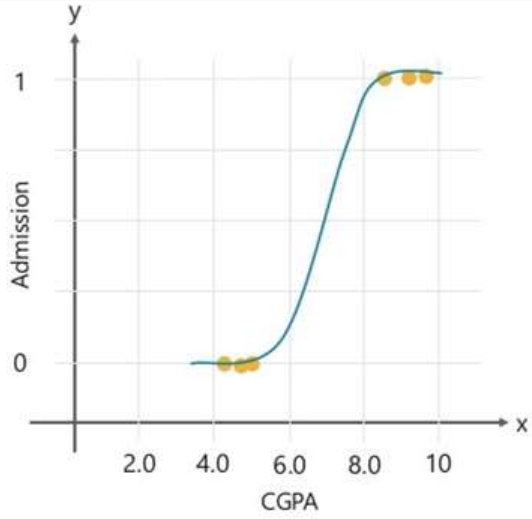
$$\log\left(\frac{Y}{1-Y}\right) = C + B1X1 + B2X2 + \ldots$$

- Y is the probability of an event to happen which you are trying to predict
- x1, x2 are the independent variables which determine the occurrence of an event i.e. Y
- C is the constant term which will be the probability of the event happening when no other factors are considered

Use case of Logistic Regression

Logistic regression can be used to predict if a student will pass or fail an exam based on the number of hours they spent studying. The dependent variable is "pass" or "fail", which are represented by the values 1 and 0, respectively.

Admission	CGPA
0	4.2
0	5.1
0	5.5
1	8.2
1	9.0
1	9.1



Type of Logistic Regression:

On the basis of the categories, Logistic Regression can be classified into three types:

1. **Binomial**: In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.
2. **Multinomial**: In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as "cat", "dogs", or "sheep"
3. **Ordinal**: In ordinal Logistic regression, there can be 3 or more possible ordered types of dependent variables, such as "low", "Medium", or "High".

Applications:

• Logistic regression is widely applied in fields such as medicine, finance, and marketing. It helps in binary classification tasks such as detecting whether an email is spam or not, predicting whether a patient has a disease like diabetes, or determining if a transaction might be fraudulent.

Example:

• Predicting Disease Occurrence: Suppose a medical researcher wants to predict the likelihood that individuals have diabetes based on their age and BMI. Here, the outcome variable Y is whether the person has diabetes (1) or not (0), and the predictors X1 and X2 are age and BMI, respectively. The logistic regression model would help estimate the probability of diabetes for different age groups and BMI levels, using historical data to determine the coefficients b1 and b2 for age and BMI.

Advantages of Logistic Regression:

- **Easy to Use**: Simple to understand and explain.
- **Good for Yes/No Questions**: Perfect for predicting things like "Will it rain?" or "Will I buy this?"
- **Gives Probabilities**: Shows how likely something is to happen.
- **Handles Curved Relationships**: Works well even if the relationship isn't straight.
- **Better with Outliers**: Less affected by extreme values than some other methods.

Disadvantages of Logistic Regression:

- **Needs Lots of Data**: Works best with a good amount of information.
- **Assumes a Simple Relationship**: It assumes a specific way the factors relate to the outcome.
- **Only for Two Options**: Mainly used for two choices; not for multiple options.
- **Struggles with Complex Patterns**: Might not capture very complicated relationships well.
- **Sensitive to Similar Variables**: If the factors are too alike, it can cause problems.

Difference Between Linear regression and Logistic Regression

	Linear Regression	Logistic Regression
1 Definition	To predict a continuous dependent variable based on values of independent variables	*To predict a categorical dependent variable based on values of independent variables*
2 Variable Type	Continuous dependent variable	Categorical dependent variable
3 Estimation method	Least square estimation	Maximum like-hood estimation
4 Equation	$Y = b_0 + b_1 x + e$	$\log \left(\frac{Y}{1 - Y} \right) = C + B1X1 + B2X2 + ...$
5 Best fit line	Straight line	Curve
6 Relationship between DV & IV	Linear relationship between the dependent and independent variable	Linear relationship is not mandatory
7 Output	Predicted integer value	Predicted binary value (0 or 1)
8 Applications	Business domain, forecasting sales	Classification problems, cybersecurity, image processing

Concept Learning

- Concept Learning can be viewed as the **task of searching** through a **large space of hypothesis** implicitly defined by hypothesis representation.
- The goal of concept learning is to **find hypothesis that best fits training example.**
- Concept learning in machine learning is the process of figuring out what a category or group of things looks like based on examples. The goal is to learn a rule or pattern that helps identify whether new things belong to the category or not.

Concept learning

MAIN POINTS:Concept: This is the category you're trying to learn. For example, if you're learning to recognize what "spam" emails look like, the concept is "spam."

- **Hypothesis:** A hypothesis is a guess or rule that tries to describe the concept. The algorithm tries different hypotheses to find the best one.
- **Examples:** You use examples to teach the algorithm. These examples are either **positive** (they belong to the category, like actual spam emails) or **negative** (they don't belong to the category, like regular emails).
- **Target Concept:** This is the real, correct rule that you want the algorithm to learn. The goal is to get as close to this as possible.

Components of Concept Learning

- **Hypothesis Space (H):**
  - The hypothesis space is the set of all possible concepts or rules that could describe the target concept. For instance, if the task is to classify objects based on color and shape, the hypothesis space includes all possible combinations of color and shape.
  - In practical terms, this space can be very large or infinite, making it computationally challenging to search through all hypotheses.
- **Generalization and Specialization:**
  - **Generalization** refers to making the hypothesis more inclusive by accepting a broader range of instances. For example, in image classification, a more generalized hypothesis might classify both small and large dogs as belonging to the "dog" category.
  - **Specialization** is the opposite, where the hypothesis is refined to accept fewer instances, becoming more specific. In the previous example, specialization might involve classifying only small dogs as "dogs."
- **Version Space:**
  - The **version space** is the subset of the hypothesis space that is consistent with the training examples. It contains all the hypotheses that correctly classify all the positive and negative examples seen so far.
  - As learning progresses, the version space shrinks as more constraints are imposed by new examples.

Concept Learning Algorithms

Several algorithms have been designed to handle concept learning. Two classical approaches are:

Find-S Algorithm (Finding a maximally Specific Hypothesis):

- The Find-S algorithm **starts with the most specific hypothesis** possible and **generalizes** it as it encounters positive examples.
- If a positive example matches the current hypothesis, no change is made. If it doesn't, the hypothesis is generalized just enough to include this new example.
- The main limitation of Find-S is that it only looks for the most specific consistent hypothesis and doesn't handle negative examples well.

Steps:

- Initialize the hypothesis to the most specific hypothesis in the hypothesis space.
- For each positive example, generalize the hypothesis to include this example, if necessary.
- Ignore negative examples.

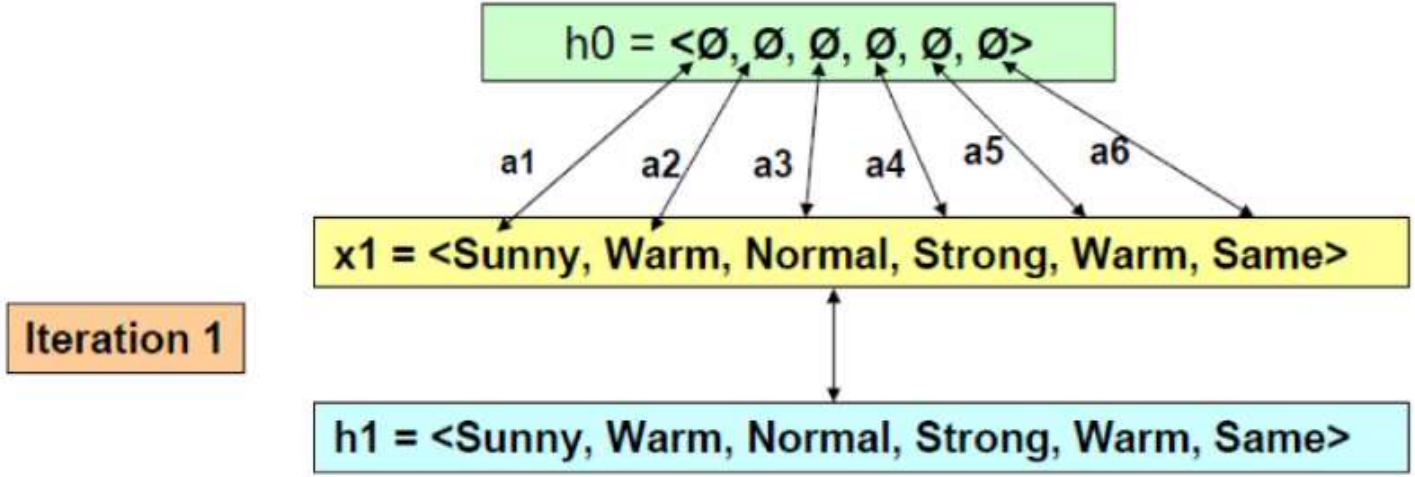
Example

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

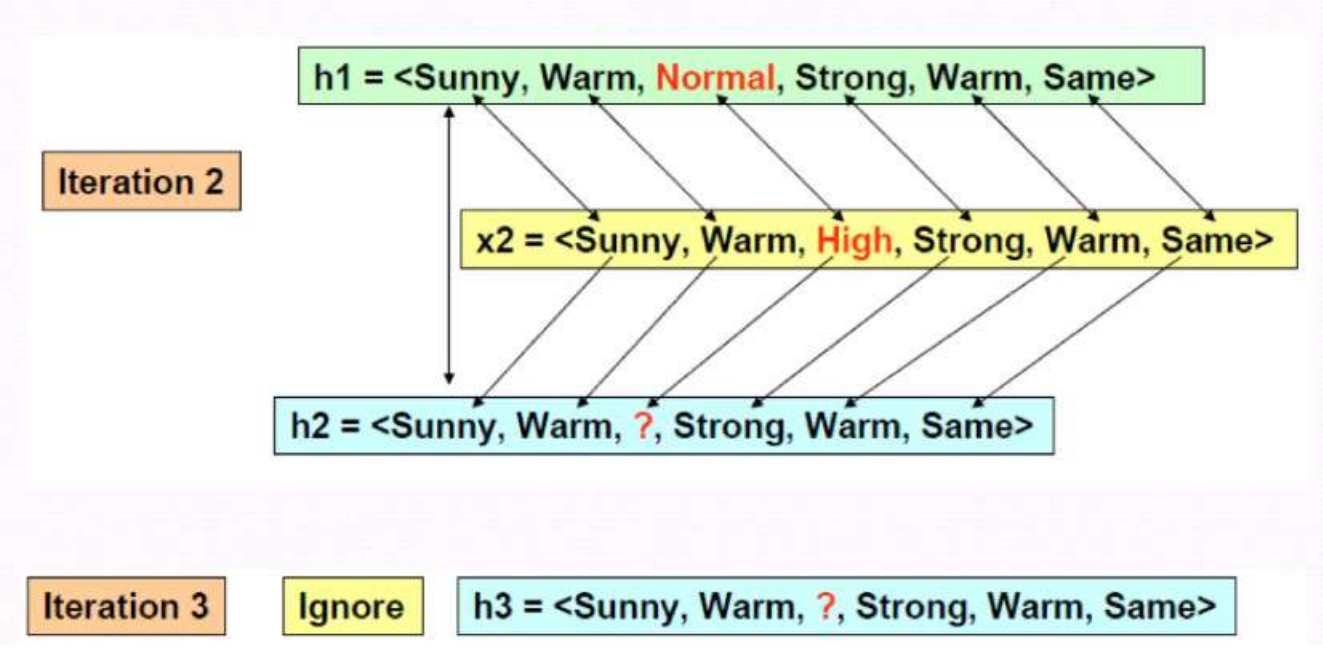
Initialize h to the most specific hypotesis in h

ho = <Ø, Ø, Ø, Ø, Ø, Ø>

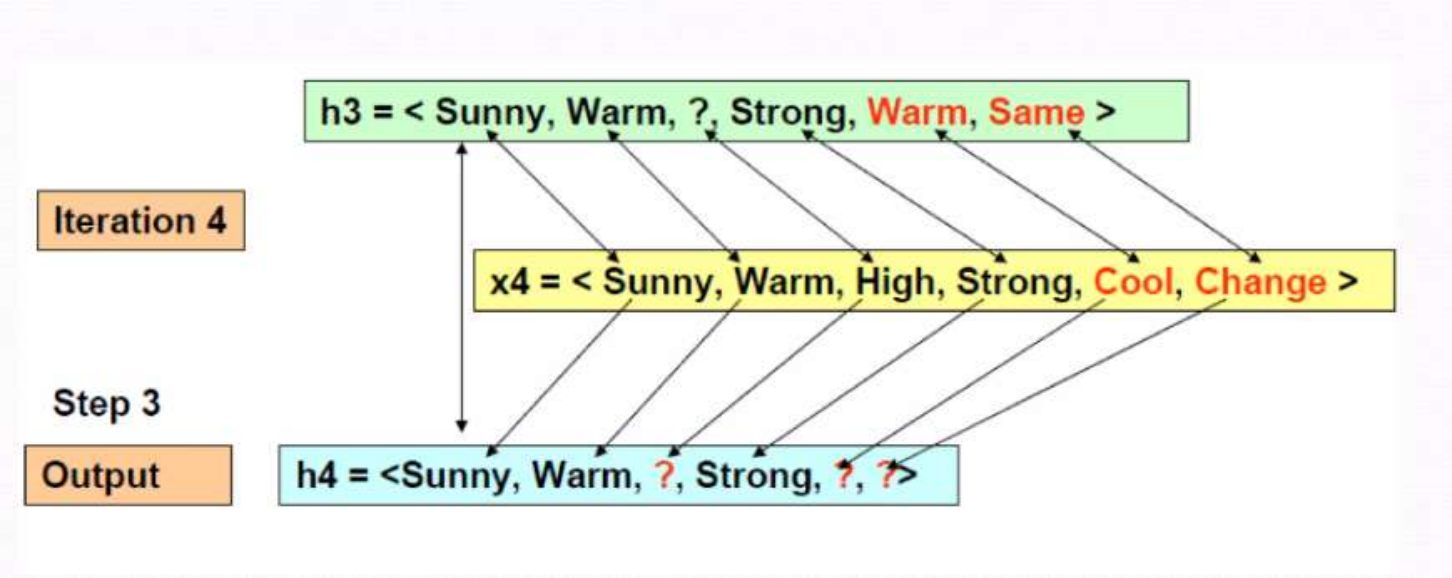
Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes



Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes



Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes



Candidate Elimination Algorithm:

- The candidate elimination algorithm maintains two boundaries in the version space: the **most general hypotheses** and the **most specific hypotheses**. It incrementally refines these boundaries as new training examples are encountered.
- When a **positive example** is seen, it eliminates hypotheses that are too **specific** (that don't include the example). When a **negative example** is seen, it eliminates hypotheses that are too **general** (that include the example).

Steps:

- Initialize the specific boundary to the most specific hypothesis and the general boundary to the most general hypothesis.
- For each positive example, generalize the specific boundary and prune the general boundary if needed.
- For each negative example, specialize the general boundary and prune the specific boundary.

Example

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Initialization

$G0 \leftarrow \{<?, ?, ?, ?, ?, ?>\}$

$S0 \leftarrow \{<\emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset>\}$

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

G0 ← {<?, ?, ?, ?, ?, ?>}

S0 ← {<Ø, Ø, Ø, Ø, Ø, Ø >}

Iteration 1

x1 = <Sunny, Warm, Normal, Strong, Warm, Same>

G1 ← {<?, ?, ?, ?, ?, ?>}

S1 ← {< Sunny, Warm, Normal, Strong, Warm, Same >}

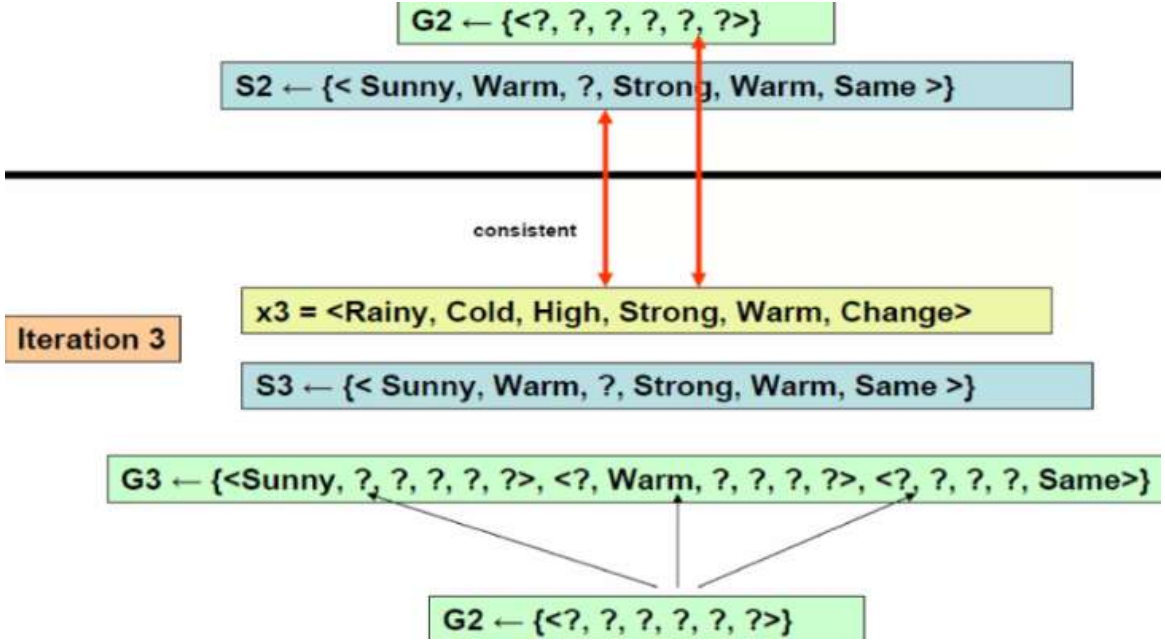
Iteration 2

x2 = <Sunny, Warm, High, Strong, Warm, Same>

G2 ← {<?, ?, ?, ?, ?, ?>}

S2 ← {< Sunny, Warm, ?, Strong, Warm, Same >}

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes



Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

S3 ← {< Sunny, Warm, ?, Strong, Warm, Same >}

G3 ← {<Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>, <?, ?, ?, ?, Same>}

Iteration 4

x4 = <Sunny, Warm, high, Strong, Cool, Change>

S4 ← {< Sunny, Warm, ?, Strong, ?, ? >}

G4 ← {<Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>}

G3 ← {<Sunny, ?, ?, ?, ?, ?>, <?, Warm, ?, ?, ?, ?>, <?, ?, ?, ?, Same>}

Challenges in Concept Learning:

- **Bias:** The learning algorithm has a certain way of searching through the possible rules. This bias affects which kind of rules it can learn.
- **Noise and Over fitting:** Real-world data often contains mistakes or randomness. Over fitting happens when the algorithm learns to fit the examples too closely, even learning the noise, instead of finding a more general rule.
- **Efficiency:** Searching through all possible rules can take a lot of time and computer power, so the algorithm needs to be efficient.

Applications:

- **Classification Tasks:** For example, sorting emails into “spam” or “not spam,” recognizing objects in pictures, or labeling texts based on their content.
- **Medical Diagnosis:** Learning to classify diseases based on patient symptoms and test results.
- **Text Analysis:** Categorizing documents or understanding the meaning behind sentences, like figuring out if a review is positive or negative.

Bayes theorem

- Bayes' theorem is a fundamental concept in **probability theory** that plays a crucial role in various machine learning algorithms, especially in the fields of Bayesian statistics and probabilistic modelling.
- It provides a way to **update probabilities based on new evidence** or information.
- In the context of machine learning, Bayes' theorem is often used in Bayesian inference and probabilistic models.

The theorem can be mathematically expressed as:

$$P(A \mid B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

where

- **P(A)** and **P(B)** are the probabilities of events A and B; also, P(B) is never equal to zero.
- **P(A|B)** is the probability of event A when event B have already occurred
- **P(B|A)** is the probability of event B when A have already occurred

Example: Pick a random card and you know its diamond. What is the probability that card is queen.

Total diamond =13,

Probability (Queen/ Diamond)= 1/13

$$P(queen/diamond) = \frac{P(diamond/queen) * P(queen)}{P(diamond)}$$

P(diamond/queen) = 1/4

P(queen) = 1/13

P(diamond) = 1/4

$$= \frac{1/4 * 1/13}{1/4}$$

$$= 1/13$$

Start from the definition of conditional probability:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \qquad P(B \mid A) = \frac{P(A \cap B)}{P(A)}.$$

Both right-hand sides equal $P(A \cap B)$, so set them equal:

$$P(A \mid B)\, P(B) = P(B \mid A)\, P(A).$$

Solve for $P(A \mid B)$:

$$\boxed{P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}}.$$

This is the Bayes' Theorem.

Here, P(A) and P(B) are probabilities of observing A and B independently of each other.

That's why we can say that they are marginal probabilities. P(B|A) and P(A|B) are conditional probabilities.

P(A) is called **Prior probability** and P(B) is called **Evidence**.

likelihood

Posterior probability

$$P(A \mid B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

prior probability

Marginal

Technical Terms of Bayes Theorem:

- *P(A/B) is the **posterior probability**- probability of hypothesis A given we have some data / evidence(B)*
- *(B/A) is the **likelihood-** probability of evidence given hypothesis is true*
- *P(A) is the **prior probability**- probability of hypothesis without observing evidence*
- *P(B) is the **total probability / Marginal-** Probability of evidence*

Steve is very shy and withdrawn, invariably helpful but with very little interest in people or in the world of reality. A meek and tidy soul, he has a need for order and structure, and a passion for detail.

Librarian

Farmer

$$P\,(\text{Librarian given description}) = \frac{4}{4+20} \approx 16.7\%$$

You have a hypothesis

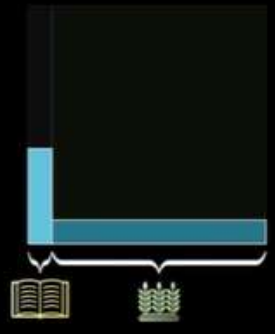
You've observed some evidence

Steve is very shy and withdrawn, invariably helpful but with very little interest in people or in the world of reality. A meek and tidy soul, he has a need for order and structure, and a passion for detail.

You want

$$P(H|E)$$

$$P\left(\begin{array}{c}\text{Hypothesis}\\ \textbf{given}\\ \text{the evidence}\end{array}\right)$$



Bayes Optimal Classifier

- The Bayes Optimal Classifier is a probabilistic model **that makes the most probable prediction for a new example**.
- It is described using the Bayes Theorem that provides a principled way for calculating a conditional probability.
- It is also closely related to the **Maximum a Posteriori**: a probabilistic framework referred to as MAP that finds the most probable hypothesis for a training dataset.
- In practice, the Bayes Optimal Classifier is computationally expensive, if not intractable to calculate, and instead, simplifications such as the Gibbs algorithm and Naive Bayes can be used to approximate the outcome.

- The **Bayes Optimal Classifier** is considered the **best possible classification method** in the context of Bayesian decision theory. It makes predictions by combining all possible hypotheses (or models) weighted by how likely each of them is given the data. In essence, it tries to find the most probable classification based on all the information available.

What It Does:

- The Bayes Optimal Classifier doesn’t rely on just one model to make predictions. Instead, it looks at all possible models or hypotheses and combines their predictions based on how likely each hypothesis is.
- It uses **Bayes' Theorem** to calculate the probability of each hypothesis being correct, given the data.
- It then averages the predictions of all these hypotheses, weighted by their probabilities.

Why It's Optimal:

- The Bayes Optimal Classifier makes the prediction that has the **highest probability of being correct.** It's called "optimal" because if you had all possible hypotheses and perfect probabilities, it would give you the best prediction that can be made.

Example:

- Let's say you want to classify an email as **spam** or **not spam**. There are many different classifiers or hypotheses you could use (e.g., a decision tree, Naive Bayes, etc.).
- The Bayes Optimal Classifier would consider **all these possible classifiers**.
- It would then calculate the probability of each classifier (hypothesis) being the correct one, based on the data you have.
- Finally, it would combine the predictions of all these classifiers, weighted by how likely each classifier is to be correct.

Maximum a Posteriori

- The Bayes Theorem, which provides a systematic means of computing a conditional probability, is used to describe it. It's also related to Maximum a Posteriori (MAP), a probabilistic framework for determining the most likely hypothesis for a training dataset.
- Take a hypothesis space that has 3 hypotheses h1, h2, and h3.
- The posterior probabilities of the hypotheses are as follows:

  P(h1/D) -> 0.4
  P(h2/D) -> 0.3
  P(h3/D) -> 0.3

- Hence, **h1 is the MAP hypothesis.** (MAP => max posterior)

Most Probable Classification

- The posterior probabilities of the hypotheses are as follows:

  P(h1/D) -> 0.4
  P(h2/D) -> 0.3
  P(h3/D) -> 0.3

- Suppose a new instance x is encountered, which is classified **negative by h2 and h3** but **positive by h1**.
- Taking all hypotheses into account, the probability that **x is positive is .4** and the probability that it is **negative is therefore .6**
- The classification generated by the MAP hypothesis is different from the **most probable classification i**n this case which is negative**.**

The most probable classification of the new instance is obtained by combining the predictions of all hypotheses, weighted by their posterior probabilities.

$$P(v_j|D) = \sum_{h_i \in H} P(v_j|h_i)P(h_i|D)$$

Where *vj* is a new instance to be classified,
H is the set of hypotheses for classifying the instance,
hi is a given hypothesis,
P(vj | hi) is the posterior probability for *vi* given hypothesis *hi*, and
P(hi | D) is the posterior probability of the hypothesis *hi* given the data *D*.

Selecting the outcome with the maximum probability is an example of a Bayes optimal classification

Bayes optimal classification:

$$\underset{v_j \in V}{\operatorname{argmax}} \sum_{h_i \in H} P(v_j|h_i)P(h_i|D)$$

Bayes optimal classification:

$$\underset{v_j \in V}{\operatorname{argmax}} \sum_{h_i \in H} P(v_j|h_i)P(h_i|D)$$

To illustrate in terms of the above example, the set of possible classifications of the new instance is $V = \{\oplus, \ominus\}$, and

$$P(h_1|D) = .4, \; P(\ominus|h_1) = 0, \; P(\oplus|h_1) = 1$$
$$P(h_2|D) = .3, \; P(\ominus|h_2) = 1, \; P(\oplus|h_2) = 0$$
$$P(h_3|D) = .3, \; P(\ominus|h_3) = 1, \; P(\oplus|h_3) = 0$$

therefore

$$\sum_{h_i \in H} P(\oplus|h_i)P(h_i|D) = .4$$
$$\sum_{h_i \in H} P(\ominus|h_i)P(h_i|D) = .6$$

and

$$\underset{v_j \in \{\oplus, \ominus\}}{\operatorname{argmax}} \sum_{h_i \in H} P(v_j|h_i)P(h_i|D) = \ominus$$

A Bayes optimal classifier is a system that classifies new cases according to Equation. This strategy increases the likelihood that the new instance will be appropriately classified.

Consider an example for Bayes Optimal Classification,
Let there be 5 hypotheses h1 through h5.

$P(h_i/D)$	$P(F/h_i)$	$P(L/h_i)$	$P(R/h_i)$
0.4	1	0	0
0.2	0	1	0
0.1	0	0	1
0.1	0	1	0
0.2	0	1	0

The MAP theory, therefore, argues that the robot should proceed forward (F). Let's see what the Bayes optimal procedure suggests.

$$\sum_{h_i \in H} P(F \mid h_i) P(h_i \mid D) = 0.4$$

$$\sum_{h_i \in H} P(L \mid h_i) P(h_i \mid D) = 0.2 + 0.1 + 0.2 = 0.5$$

$$\sum_{h_i \in H} P(R \mid h_i) P(h_i \mid D) = 0.1$$

Thus, the Bayes optimal procedure recommends the robot turn left.

Naïve Bayes classifier

- **Naive Bayes** is a simple and fast algorithm used for classification, based on **Bayes' Theorem**. It's one of the easiest **supervised learning** methods and works well even with large amounts of data.

- The Naive Bayes classifier assumes that each feature (like income, age, or location) affects the outcome **independently** of other features. For example, when deciding if a person is a good candidate for a loan, factors like income, age, and credit history are considered separately, even if they are related. This is why it's called "naive"—because it assumes no connection between features, which simplifies the calculations.

Bayes theorem is cornerstone for **Naive Bayes classifier** because it provides a way to calculate the posterior probability P(h|d) from prior probability P(h) together with P(d) and P(d|h)

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

$$\begin{aligned} h_{MAP} &\equiv \underset{h \in H}{\operatorname{argmax}} P(h|D) \\ &= \underset{h \in H}{\operatorname{argmax}} \frac{P(D|h)P(h)}{P(D)} \\ &= \underset{h \in H}{\operatorname{argmax}} P(D|h)P(h) \end{aligned}$$

Bayesian approach to classify new instance is to assign the most probable target value, V_{MAP} given the attribute values <a_1, a_2.....a_n> that describe the instance

$$v_{MAP} = \underset{v_j \in V}{\operatorname{argmax}} P(v_j | a_1, a_2 \ldots a_n)$$

We can use Bayes theorem to rewrite this expression as

$$v_{MAP} = \underset{v_j \in V}{\operatorname{argmax}} \frac{P(a_1, a_2 \ldots a_n | v_j) P(v_j)}{P(a_1, a_2 \ldots a_n)}$$

$$= \underset{v_j \in V}{\operatorname{argmax}} P(a_1, a_2 \ldots a_n | v_j) P(v_j)$$

$$h_{MAP} \equiv \underset{h \in H}{\operatorname{argmax}} P(h|D)$$

$$= \underset{h \in H}{\operatorname{argmax}} \frac{P(D|h) P(h)}{P(D)}$$

$$= \underset{h \in H}{\operatorname{argmax}} P(D|h) P(h)$$

Naive Bayes classifier:

$$v_{NB} = \underset{v_j \in V}{\operatorname{argmax}} P(v_j) \prod_i P(a_i | v_j)$$

Example

Day	Outlook	Temp	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

- New instance to be classified

$$< Outlook = Sunny, Temp = Cool, Humidity = High, Wind = Strong >$$

- Naïve Bayes Classifier

$$v_{MAP} = \underset{v_j \in V}{\operatorname{argmax}} \frac{P(a_1, a_2 \ldots a_n | v_j) P(v_j)}{P(a_1, a_2 \ldots a_n)} = \underset{v_j \in V}{\operatorname{argmax}} P(a_1, a_2 \ldots a_n | v_j) P(v_j)$$

- $v_{NB} = \underset{v_j \in V}{\operatorname{argmax}}\, p(v_j) \prod_i p(a_i | v_j)$

$$p(yes|new\ Instance) = p(yes) * p(sunny|yes) * p(cool|yes) * p(high|yes) * p(strong|yes)$$

$$p(no|new\ Instance) = p(no) * p(sunny|no) * p(cool|no) * p(high|no) * p(strong|no)$$

Day	Outlook	Temp	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

- $p(yes) = \frac{9}{14}$ $and\ p(no) = \frac{5}{14}$

Likelihood table

Outlook	Yes	No
Sunny	2/9	3/5
Overcast	4/9	0/5
Rainy	3/9	3/5

Temp	Yes	No
Hot	2/9	3/5
Mild	4/9	2/5
Cool	3/9	2/5

Humidity	Yes	No
High	3/9	4/5
Normal	6/9	1/5

Wind	Yes	No
Strong	3/9	3/5
Weak	6/9	2/5

$$p(yes|new\ Instance) = p(yes) * p(sunny|yes) * p(cool|yes) * p(high|yes) * p(strong|yes)$$

$$p(no|new\ Instance) = p(no) * p(sunny|no) * p(cool|no) * p(high|no) * p(strong|no)$$

Day	Outlook	Temp	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

- $p(yes) = \frac{9}{14}$ and $p(no) = \frac{5}{14}$

Outlook	Yes	No
Sunny	2/9	3/5
Overcast	4/9	0/5
Rainy	3/9	3/5

Temp	Yes	No
Hot	2/9	3/5
Mild	4/9	2/5
Cool	3/9	2/5

Humidity	Yes	No
High	3/9	4/5
Normal	6/9	1/5

Wind	Yes	No
Strong	3/9	3/5
Weak	6/9	2/5

PlayTennis = No

$$p(yes|new\ Instance) = p(yes) * p(sunny|yes) * p(cool|yes) * p(high|yes) * p(strong|yes) = 0.0053$$

$$p(no|new\ Instance) = p(no) * p(sunny|no) * p(cool|no) * p(high|no) * p(strong|no) = 0.0206$$

Advantages

- It is not only a simple approach but also a fast and accurate method for prediction.
- Naive Bayes has a very low computation cost.
- It can efficiently work on a large dataset.
- It performs well in case of discrete response variable compared to the continuous variable.
- It can be used with multiple class prediction problems.
- It also performs well in the case of text analytics problems.
- When the assumption of independence holds, a Naive Bayes classifier performs better compared to other models like logistic regression.

Disadvantages

- The assumption of independent features. In practice, it is almost impossible that model will get a set of predictors which are entirely independent.
- If there is no training tuple of a particular class, this causes zero posterior probability. In this case, the model is unable to make predictions. This problem is known as Zero Probability/Frequency Problem.

Bayesian Belief Networks

- Bayesian belief networks (BBNs) are probabilistic graphical models that are used to represent uncertain knowledge and make decisions based on that knowledge.
- A Bayesian network represents a **set of variables** and their **conditional dependencies** through a ***directed acyclic graph (DAG)***.
- The nodes in the graph represent variables, while the edges indicate the probabilistic/ causal relationships between them.

- Bayesian belief networks are particularly suited to the target recognition problem, where the category, identity and class of a target track are to be determined.

Bayesian Network Consists Of Two Parts:

Directed Acyclic Graph

- This is a graphical representation of the variables in the network and the causal relationships between them.
- The nodes in the DAG represent variables, and the edges represent the dependencies between the variables.
- The arrows in the graph indicate the direction of causality.

Table of Conditional Probabilities

- For each node in the DAG, there is a corresponding table of conditional probabilities that specifies the probability of each possible value of the node given the values of its parents in the DAG.
- These tables encode the probabilistic relationships between the variables in the network.

Together, the DAG and the conditional probability tables allow us to perform probabilistic inference in the network, such as computing the probability of a particular variable given the values of other variables in the network

Example

1. To prevent break-ins, Harry put a brand-new burglar alarm at his house.
2. The alarm consistently reacts to a break-in, but it also reacts to little earthquakes.
3. David and Sopia, two of Harry's neighbours, have agreed to call Harry at work when they hear the alarm.
4. David always calls Harry when the alarm goes off, but occasionally he confuses phone ringing with alarm and calls too.
5. Sopia, on the other hand, enjoys listening to loud music, so occasionally she doesn't hear the alarm. Here, we'd want to calculate the likelihood of a burglar alarm.

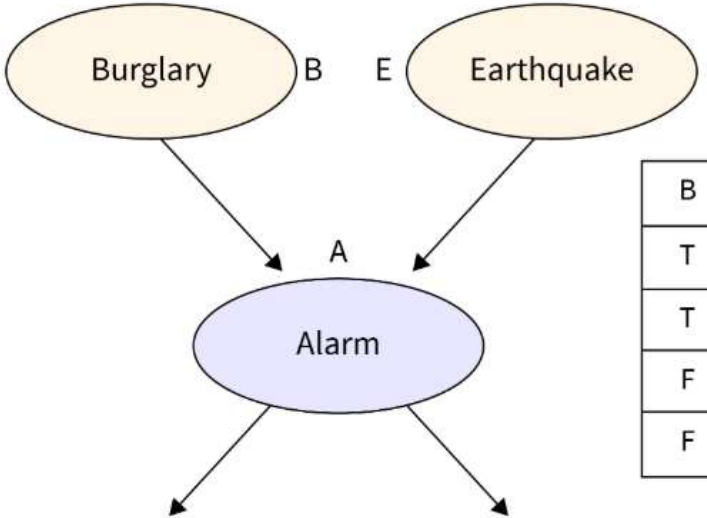
Problem:

Determine the probability that the alarm went off but that neither a burglary nor an earthquake had taken place, and that both David and Sopia had called Harry.

T	0.02
F	0.998

P(B)

P(~B)



P(E)

P(~E)

T	0.001
F	0.999

B	E	P(A=T)	P(A=F)
T	T	0.94	0.06
T	F	0.95	0.04
F	T	**0.31**	0.69
F	F	**0.001**	0.999

A	P(D=T)	P(D=F)
T	0.91	0.09
F	0.05	0.95

A	P(S=T)	P(S=F)
T	0.75	0.25
F	0.02	0.98

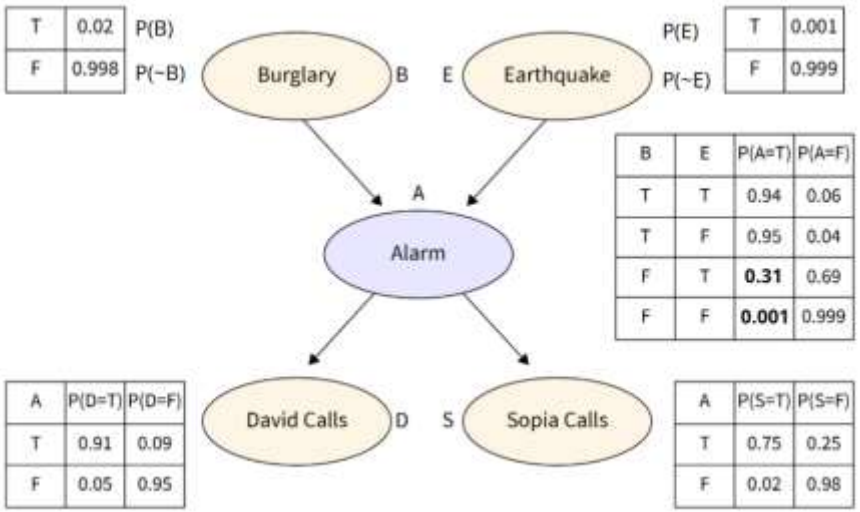
Determine the probability that the alarm went off but that neither a burglary nor an earthquake had taken place, and that both David and Sopia had called Harry.

$P(D \wedge S \wedge A \wedge \sim B \wedge \sim E) =$

$P(D|A)\ P(S|A)\ P(A|\sim B \sim E)\ P(\sim B)\ P(\sim E)$

$= 0.91 X\ 0.75\ X\ 0.001\ X\ 0.998\ X\ 0.999$

$=0.00068045$



T	0.02	P(B)
F	0.998	P(~B)

T	0.001	P(E)
F	0.999	P(~E)

B	E	P(A=T)	P(A=F)
T	T	0.94	0.06
T	F	0.95	0.04
F	T	**0.31**	0.69
F	F	**0.001**	0.999

A	P(D=T)	P(D=F)
T	0.91	0.09
F	0.05	0.95

A	P(S=T)	P(S=F)
T	0.75	0.25
F	0.02	0.98

Advantages:

- Intuitive, graphical, and efficient
- Accounts for sources of uncertainty
- Allows for information updating
- Models multiple interdependencies
- Models distributed & interacting systems
- Identifies critical components & cut sets
- Includes utility and decision nodes

Disadvantages:

- Not ideally suited for computing small probabilities
- Practical limitations on the type of distributions and the form of statistical dependence
- Computationally demanding for systems with a large number of random variables
- Exponential growth of computational effort with increased number of states

Expectation - Maximization Algorithm

- In the real-world applications of machine learning, it is very common that there are many relevant features available for learning but only a **small subset of them are observable.**
- The *Expectation-Maximization algorithm* can be used to **find the latent variables** (variables that are not directly observable and are actually inferred from the values of the other observed variables).
- This algorithm is actually the base for many unsupervised clustering algorithms in the field of machine learning.
- The **Expectation-Maximization (EM) algorithm** is a powerful tool in machine learning and statistics, used for finding the maximum likelihood estimates of parameters in probabilistic models, especially when the data is incomplete, has missing values, or involves latent (hidden) variables. The EM algorithm is iterative, and it alternates between two main steps: **Expectation (E-step)** and **Maximization (M-step)**, hence the name.

Working of EM Algorithm

1. Initially, a set of initial values of the parameters are considered. A set of incomplete observed data is given to the system with the assumption that the observed data comes from a specific model.

2. The next step is known as “Expectation” – step or ***E-step***. In this step, we use the observed data in order to estimate or guess the values of the missing or incomplete data. It is basically used to update the variables.

3. The next step is known as “Maximization” – step or ***M-step***. In this step, we use the complete data generated in the preceding “Expectation” – step in order to update the values of the parameters. It is basically used to update the hypothesis.

4. Now, in the fourth step, it is checked whether the values are converging or not, if yes, then stop, otherwise repeat ***step-2*** and ***step-3*** i.e. “Expectation” – step and “Maximization” – step until the convergence occurs

Algorithm Steps

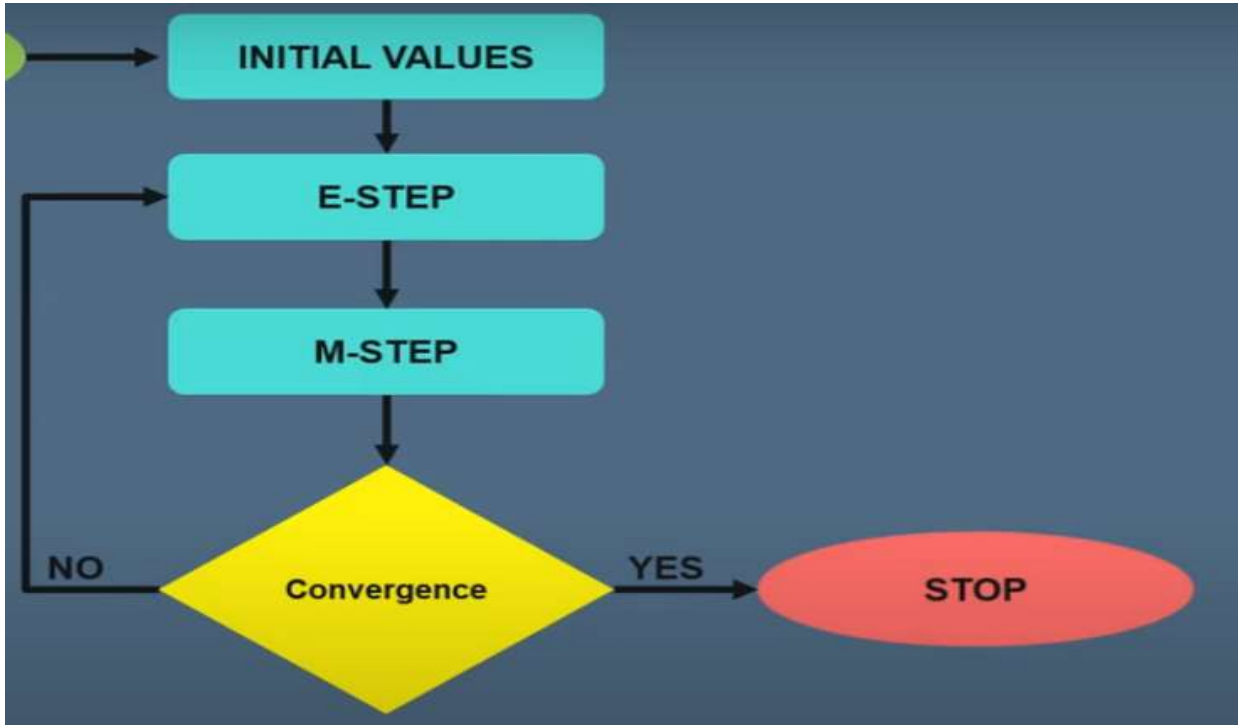
1. Given a set of incomplete data, consider a set of starting parameters.

2. **Expectation step (E – step):** Using the observed available data of the dataset, estimate (guess) the values of the missing data.

3. **Maximization step (M – step):** Complete data generated after the expectation (E) step is used in order to update the parameters.

4. Repeat step 2 and step 3 until convergence

How does it works



Usage of EM Algorithm

- It can be used to fill the missing data in a sample.
- It can be used as the basis of unsupervised learning of clusters.
- It can be used for the purpose of estimating the parameters of Hidden Markov Model (HMM).
- It can be used for discovering the values of latent variables.

Advantages of EM Algorithm

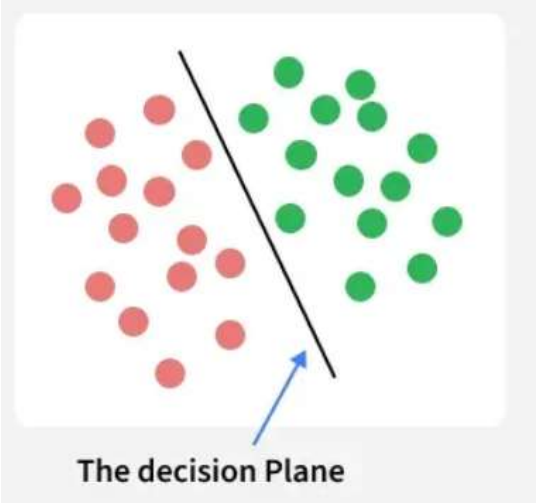
- It is always guaranteed that likelihood will increase with each iteration.
- The E-step and M-step are often pretty easy for many problems in terms of implementation.
- Solutions to the M-steps often exist in the closed form.

Disadvantages of EM Algorithm

- It has slow convergence.
- It makes convergence to the local optima only.
- It requires both the probabilities, forward and backward (numerical optimization requires only forward probability).

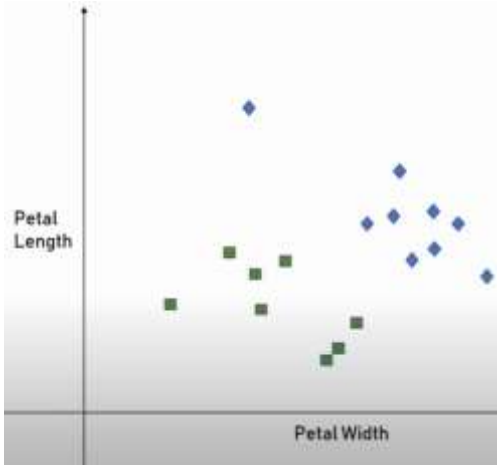
Support Vector Machine (SVM)

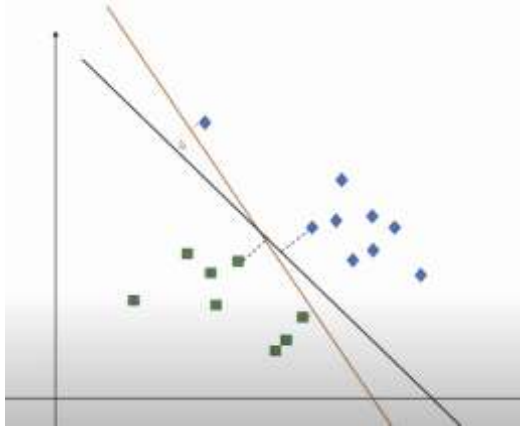
- Support Vector Machine or SVM is one of the most popular **Supervised Learning** algorithms, which is used for **Classification** as well as **Regression** problems.
- However, primarily, it is used for **Classification** problems in Machine Learning.
- It works by finding **best line or decision boundary** that can segregate n-dimensional space into classes, so that we can easily put the new data point in the correct category in the future.
- This best decision boundary is called a **hyperplane**.



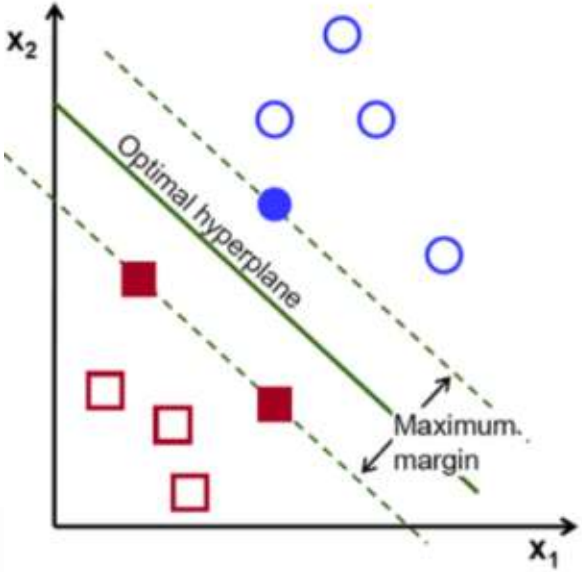
Hyperplane

- There can be multiple lines / decision boundaries to segregate classes in n-dimensional space, but we need to find the **best** decision boundary that helps to classify the data points.
- This best decision boundary is called hyperplane of the SVM

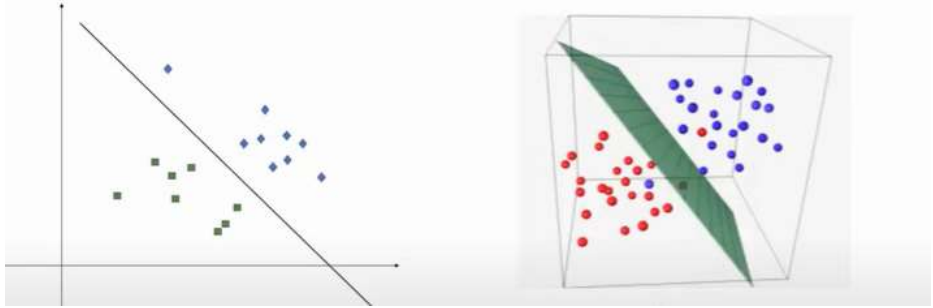




- We always create a hyperplane that has a **maximum margin**, which means the maximum distance between the data points.
- **Margin**: The margin is the perpendicular distance between the closest data points and the hyperplane from each class.
    - SVM tries to make this margin as wide as possible, which helps the model be more accurate when classifying new data.
    - The best optimised line (hyperplane) with maximum margin is termed as **maximum margin hyperplane.**
- The dimensions of the hyperplane depend on the features present in the dataset.
- If there are 2 features (as shown in image), then the hyperplane will be a **straight line**.
- If there are 3 features, then the hyperplane will be a **2-dimensional plane**.
- For n features we have n-1 dimensional plane.

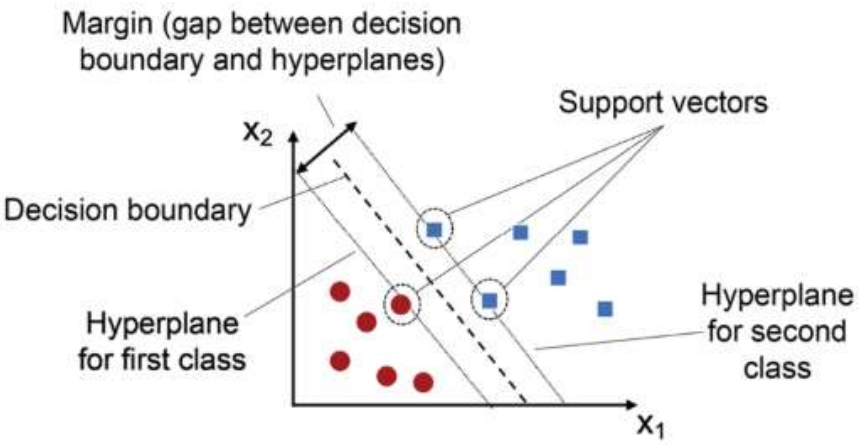


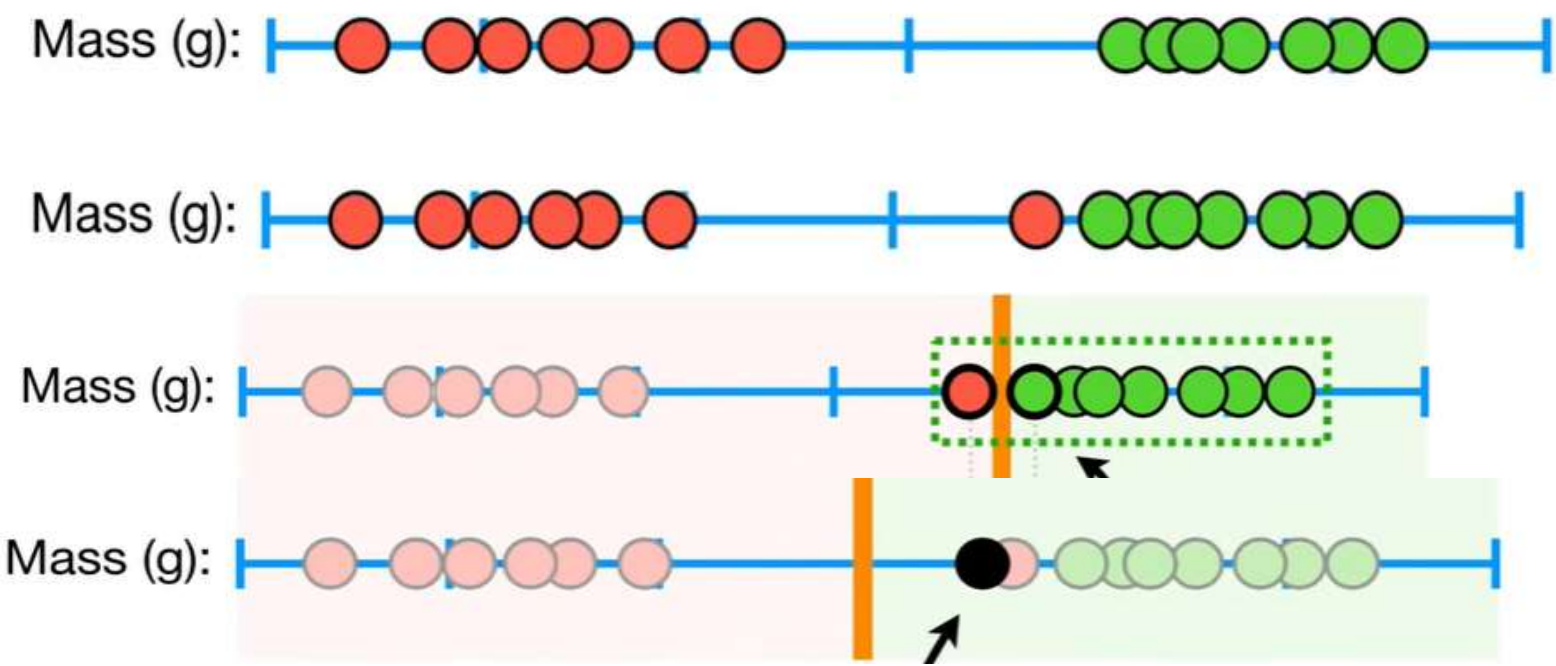
2D 3D



Support Vectors

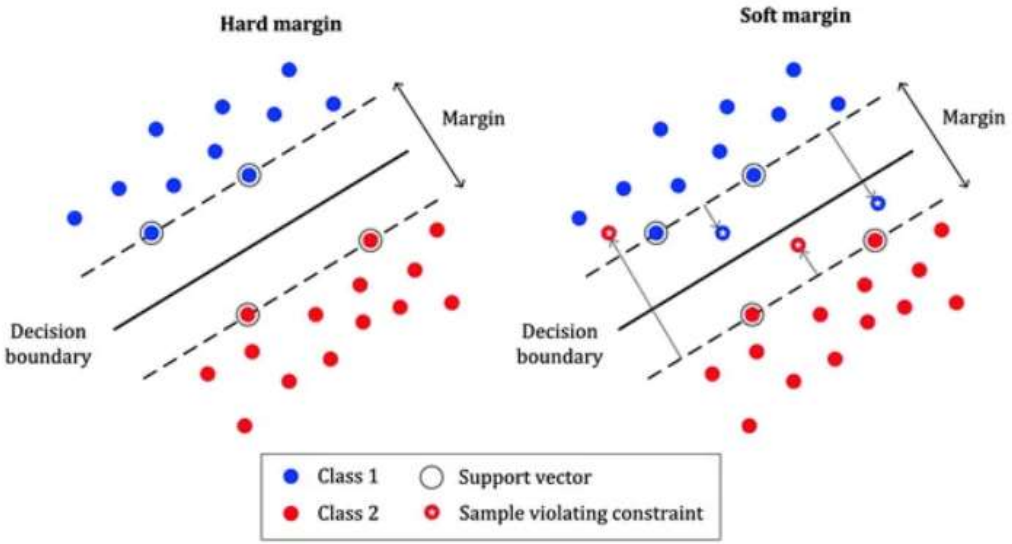
- The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as **Support Vectors**.
- Since these vectors support the hyperplane, they are called **Support Vectors**.
- These points are important because they help determine where the hyperplane should be. If you change or remove other points, it won't affect the position of the hyperplane as long as the support vectors stay the same.





When we allow misclassifications, the distance between observation and the threshold is called soft margin

- **Hard Margin**: Assumes data is perfectly separable by the hyperplane. All points must lie outside the margin.
- **Soft Margin**: Allows some misclassification or margin violations for non-linearly separable data.
- SVM is a discriminative classifier which intakes the training data, the algorithm outputs an optimal hyperplane which categorizes new examples



Example

Consider the following data points and find the optimal hyperplane for these data points: (3,1) (3,-1) (6,1) (6,-1) (1,0) (0,1) (0,-1) (-1,0)

Step1: Plot all the points on plane.

Step2: Find the support vectors.

Step3: Add the bias (b) value in vectors to convert 2D to 3D form.

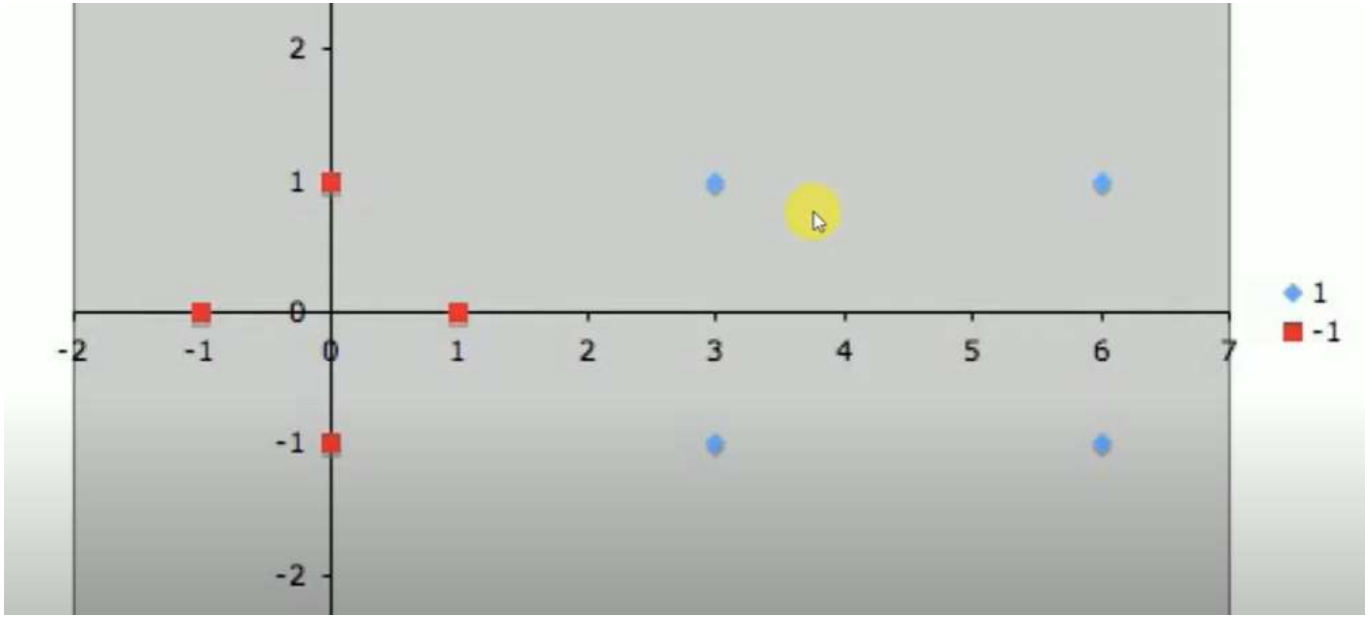
Step4: Consider Parameters for every support vector like α1, α2 etc.

Step5: Find the linear equations and solve them to find parameter values.

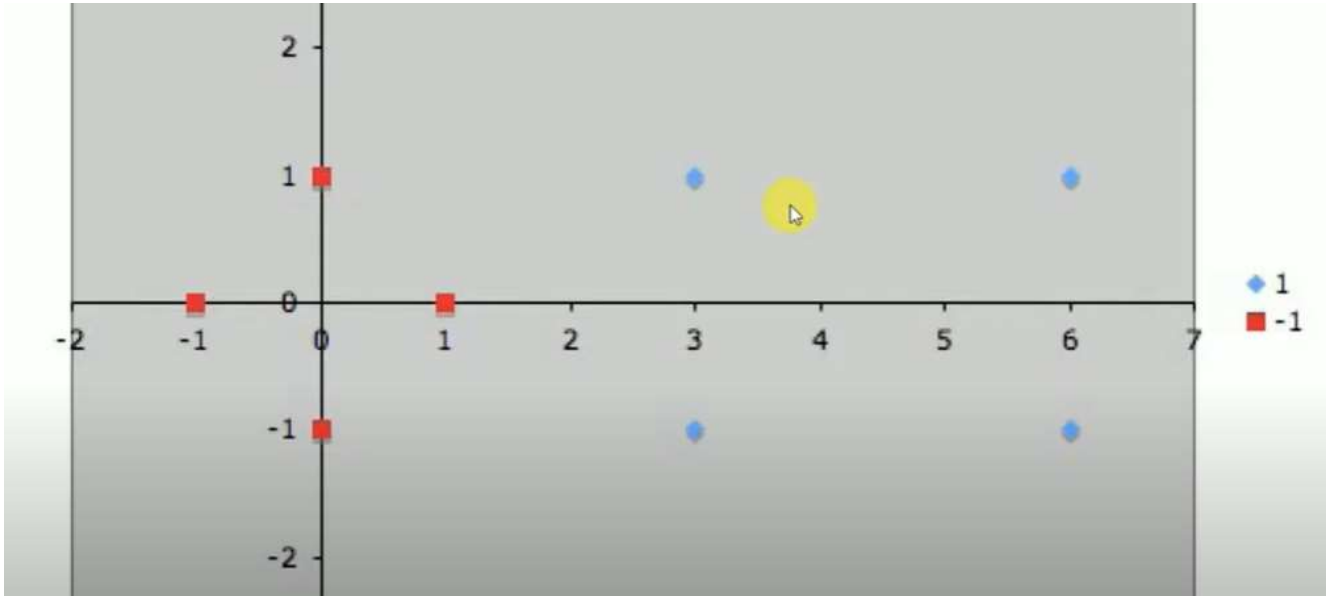
Step6: Final equation for hyperplane.

Step1: Plot all the points on plane.

Data points: (3,1) (3,-1) (6,1) (6,-1) (1,0) (0,1) (0,-1) (-1,0)



Step2: Find the support vectors



Support Vectors Identified: (1,0) (3,1) (3,-1)

Step3: Add the bias (b) value in vectors to convert 2D to 3D form.

Each vector is augmented with 1 as bias input.

Support Vectors Identified: (1,0) (3,1) (3,-1)

So, $s_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, then $\tilde{s_1} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}$

Similarly,

$s_2 = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$, then $\tilde{s_2} = \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix}$ *and* $s_3 = \begin{pmatrix} 3 \\ -1 \end{pmatrix}$, then $\tilde{s_3} = \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix}$

Step4: Consider Parameters for every support vector like α1, α2 etc.

$$\tilde{s_1} = \begin{pmatrix}1\\0\\1\end{pmatrix} \quad \tilde{s_2} = \begin{pmatrix}3\\1\\1\end{pmatrix} \quad \tilde{s_3} = \begin{pmatrix}3\\-1\\1\end{pmatrix}$$

$$\begin{aligned}
\alpha_1 \tilde{s_1} \cdot \tilde{s_1} + \alpha_2 \tilde{s_2} \cdot \tilde{s_1} + \alpha_3 \tilde{s_3} \cdot \tilde{s_1} &= -1 \\
\alpha_1 \tilde{s_1} \cdot \tilde{s_2} + \alpha_2 \tilde{s_2} \cdot \tilde{s_2} + \alpha_3 \tilde{s_3} \cdot \tilde{s_2} &= +1 \\
\alpha_1 \tilde{s_1} \cdot \tilde{s_3} + \alpha_2 \tilde{s_2} \cdot \tilde{s_3} + \alpha_3 \tilde{s_3} \cdot \tilde{s_3} &= +1
\end{aligned}$$

$$\alpha_1 \begin{pmatrix}1\\0\\1\end{pmatrix}\begin{pmatrix}1\\0\\1\end{pmatrix} + \alpha_2 \begin{pmatrix}3\\1\\1\end{pmatrix}\begin{pmatrix}1\\0\\1\end{pmatrix} + \alpha_3 \begin{pmatrix}3\\-1\\1\end{pmatrix}\begin{pmatrix}1\\0\\1\end{pmatrix} = -1$$

$$\alpha_1 \begin{pmatrix}1\\0\\1\end{pmatrix}\begin{pmatrix}3\\1\\1\end{pmatrix} + \alpha_2 \begin{pmatrix}3\\1\\1\end{pmatrix}\begin{pmatrix}3\\1\\1\end{pmatrix} + \alpha_3 \begin{pmatrix}3\\-1\\1\end{pmatrix}\begin{pmatrix}3\\1\\1\end{pmatrix} = 1$$

$$\alpha_1 \begin{pmatrix}1\\0\\1\end{pmatrix}\begin{pmatrix}3\\-1\\1\end{pmatrix} + \alpha_2 \begin{pmatrix}3\\1\\1\end{pmatrix}\begin{pmatrix}3\\-1\\1\end{pmatrix} + \alpha_3 \begin{pmatrix}3\\-1\\1\end{pmatrix}\begin{pmatrix}3\\-1\\1\end{pmatrix} = 1$$

Step5: Find the linear equations and solve them to find parameter values.

$$\alpha_1(1+0+1)+\alpha_2(3+0+1)+\alpha_3(3+0+1) = -1$$
$$\alpha_1(3+0+1)+\alpha_2(9+1+1)+\alpha_3(9-1+1) = 1$$
$$\alpha_1(3+0+1)+\alpha_2(9-1+1)+\alpha_3(9+1+1) = 1$$

$$2\alpha_1 + 4\alpha_2 + 4\alpha_3 = -1$$
$$4\alpha_1 + 11\alpha_2 + 9\alpha_3 = 1$$
$$4\alpha_1 + 9\alpha_2 + 11\alpha_3 = 1$$

$$\alpha_1 = -3.5$$
$$\alpha_2 = 0.75$$
$$\alpha_3 = 0.75$$

Step6: Final equation for hyperplane.

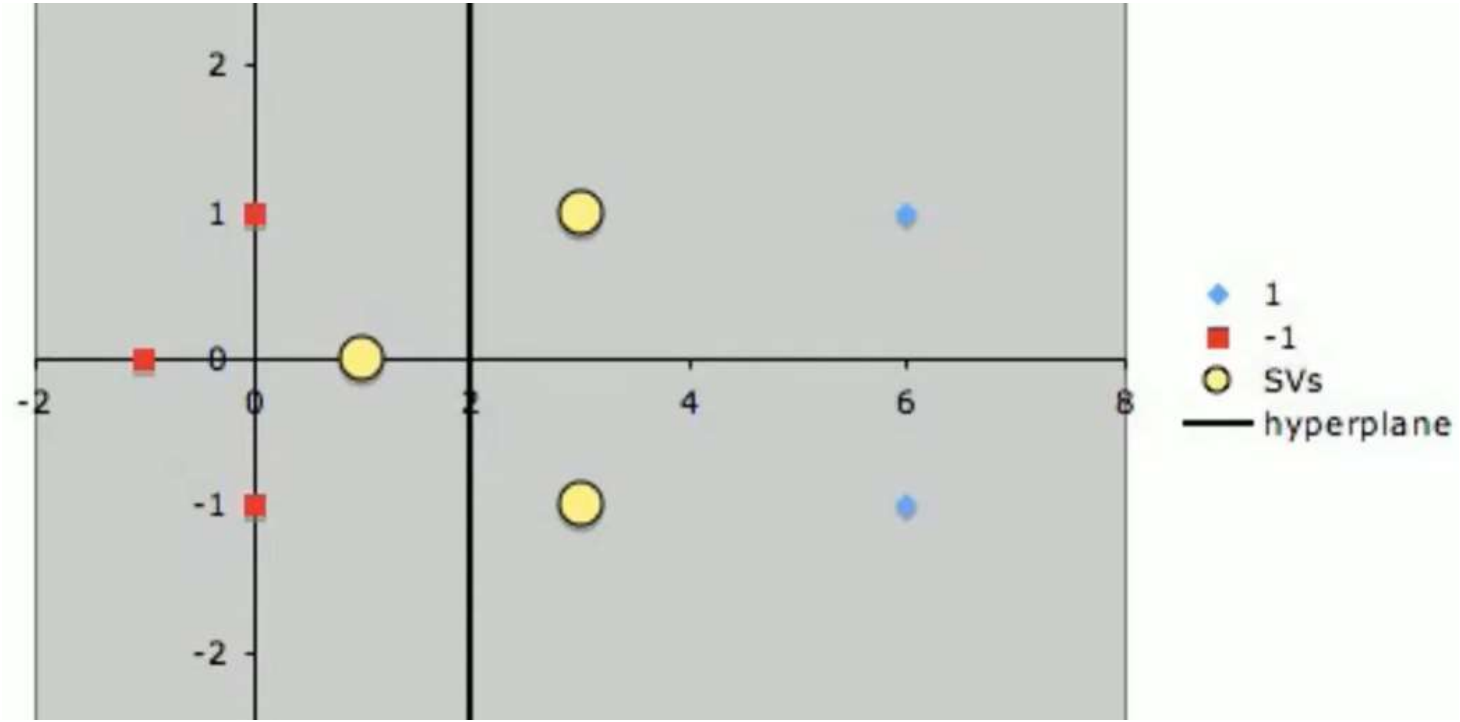
$$
\begin{aligned}
\tilde{w} &= \sum_i \alpha_i \tilde{s}_i \\
&= -3.5 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + 0.75 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + 0.75 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \\
&= \begin{pmatrix} 1 \\ 0 \\ -2 \end{pmatrix}
\end{aligned}
$$

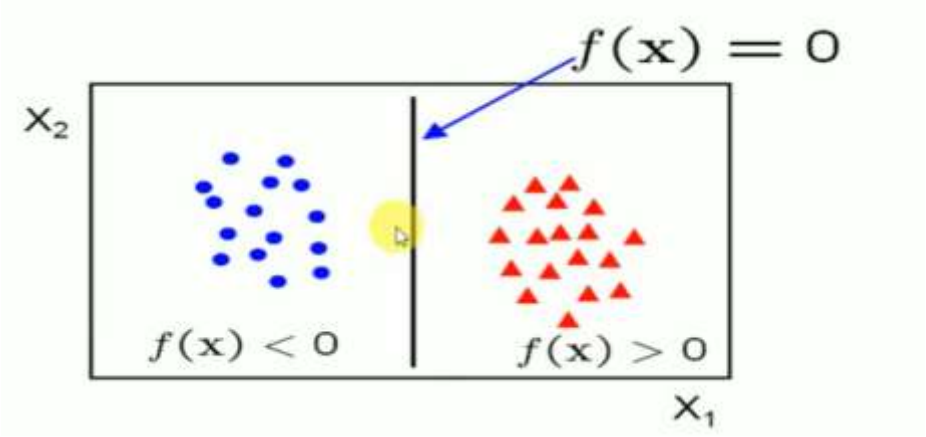
- Finally, remembering that our vectors are augmented with a bias.
- We can equate the last entry in $\tilde{w}$ as the hyperplane offset b and write the separating
- Hyperplane equation $\boldsymbol{y} = \boldsymbol{wx} + \boldsymbol{b}$
- with $\boldsymbol{w} = \binom{1}{0}$ and $\boldsymbol{b} = -2$.

If w=(1,0) hyperplane is parallel to y axis.

If w=(0,1) hyperplane is parallel to x axis.

If w=(1,0) hyperplane is at 45 degree angle wrt x and y axis.


2
1
0
-1
-2
-2
0
2
4
6
8
1
-1
SVs
hyperplane

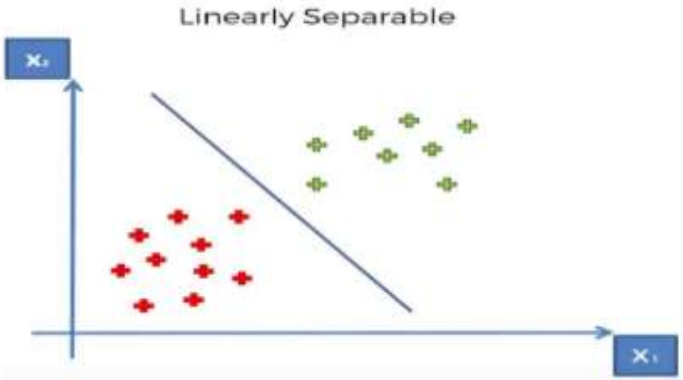


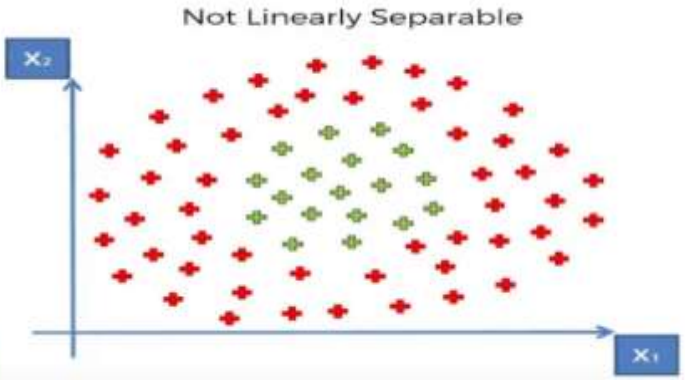
Equation of hyperplane f(x) = wx+b

Types of SVM

Linear SVM: Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.

Non-linear SVM: Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

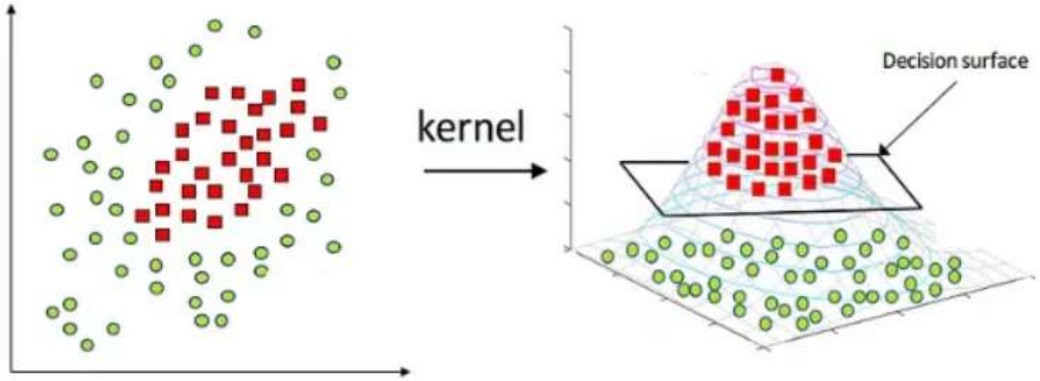




Kernel

•If the data isn't easily separable linearly SVM can use techniques called **kernel functions t**o transform the data into a higher dimension, making it easier to separate.

•A kernel function transforms data into higher-dimensional space to make it linearly separable.



Input Space

Feature Space

For example, one mapping function ϕ: $R^2 \rightarrow R^3$ used to transform a 2D data to 3D data is given as follows:

$$\phi(x, y) = (x^2, \sqrt{2}xy, y^2)$$

Consider a point (2, 3) in 2D space, if you apply above mapping function we can convert it into 3D space and it looks like this,

Here x=2 y=3

Hence point in 3D space is: $(2^2, \sqrt{2} \cdot 2 \cdot 3, 3^2) = (4, 6\sqrt{2}, 9)$

- While **mapping functions** play an important role, there are many disadvantages, as mapping involves **more computations and learning costs**.

- Also, the disadvantages of transformations are that there is **no generalized thumb rule** available describing what transformations should be applied and if the data is large, mapping process takes huge amount of time.

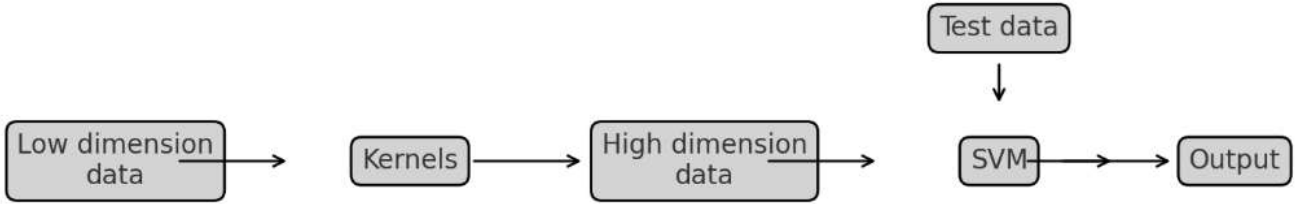
- In real applications, there might be many features in the data and applying transformations that involve many polynomial combinations of these features will lead to extremely high and impractical computational costs.

- In this context, only kernels are useful.

- Kernels are used to compute the value without transforming the data.

What is a Kernel?

- Kernels are a set of functions used to transform data from lower dimension to higher dimension and to manipulate data using dot product at higher dimensions.
- Kernels help separate data that isn't linearly separable.
- They allow SVMs to find better decision boundaries by projecting data into higher dimensions.
- The use of kernels is to apply transformation to data and perform classification at the higher dimension as shown in Figure.



Kernel Trick

- The kernel trick is a powerful technique to handle non-linear data by implicitly mapping it into a high-dimensional feature space without actually computing the high-dimensional coordinates.
- Instead of calculating the data points' coordinates in the new space, the kernel trick uses a **kernel function to compute the dot product (inner product) of these mapped points directly from the original data.**
- This approach avoids the significant computational cost of explicit transformation and thus saves time, making complex, non-linear patterns linearly separable and solvable with linear models

$$\phi(x, y) = (x^2, \sqrt{2}xy, y^2)$$

$$\phi(\mathbf{a})^T \cdot \phi(\mathbf{b}) = \begin{pmatrix} a_1{}^2 \\ \sqrt{2}\, a_1 a_2 \\ a_2{}^2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1{}^2 \\ \sqrt{2}\, b_1 b_2 \\ b_2{}^2 \end{pmatrix} = a_1{}^2 b_1{}^2 + 2a_1 b_1 a_2 b_2 + a_2{}^2 b_2{}^2$$

$$= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = \left(\mathbf{a}^T \cdot \mathbf{b} \right)^2$$

- **Kernel Trick**:
  - Computes relationships in higher-dimensional space without actual transformation.
  - Saves computation time and resources.
- **Benefits**:
  - Higher accuracy and lower errors in classification.
  - Helps in finding hyperplanes that effectively separate classes.
- **Common Kernel Functions**:

1. Linear kernel
2. Polynomial kernel
3. Gaussian kernel

- Using the right kernel can greatly enhance SVM performance on complex datasets!

Linear Kernel

- It is the most basic type of kernel, usually one dimensional in nature. It proves to be the best function when there are lots of features. The linear kernel is mostly preferred for **text-classification problems** as most of these kinds of classification problems can be linearly separated.
- Linear kernel functions are **faster** than other functions.

- **Linear Kernel Formula** $$K(x, y) = x \cdot y$$
- It is then applied to any combination of two data points (samples) in the dataset. The dot product of the two points determines the **cosine_similarity** between both points. The higher the value, the more similar the points are.

$$K(x, y) = x \cdot y$$

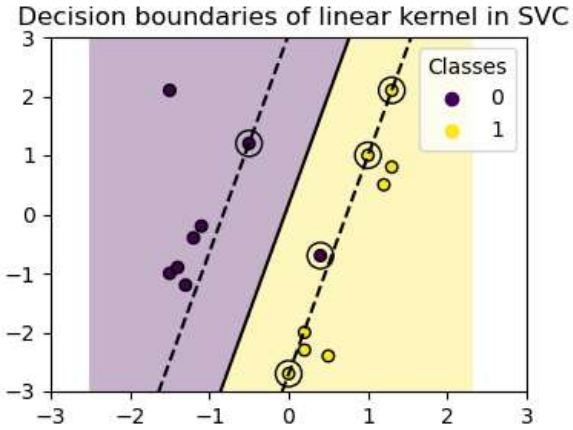
That means, for two feature vectors, the kernel gives a measure of how aligned (similar) they are.

- If vectors point in the same direction → dot product is large and positive.
- If vectors are orthogonal → dot product is 0.
- If vectors point in opposite directions → dot product is negative.

So the SVM uses this dot product as the similarity score to build its decision boundary (hyperplane).

$$x \cdot y = x^T y$$

- It is used when the data is Linearly separable, that is, it can be separated using a single Line. It is one of the most common kernels to be used. It is mostly used when there are a Large number of Features in a particular Data Set.
- Training a Support Vector Classification on a linear kernel results in an **untransformed feature space**, where the hyperplane and the margins are straight lines. Due to the lack of expressivity of the linear kernel, the trained classes do not perfectly capture the training data.



List of applications for the linear kernel:

- **Text Classification**: Used for spam detection and document categorization.
- **Image Classification**: Effective for classifying simple images.
- **Natural Language Processing**: Helps with sentiment analysis.
- **Bioinformatics**: Classifies genes and proteins.
- **Stock Market Predictions**: Models linear trends in financial data.
- **Handwriting Recognition**: Works for recognizing simple handwriting.
- **Recommendation Systems**: Analyzes user-item interactions.

Polynomial Kernel

- A polynomial kernel maps data into a higher-dimensional space using a polynomial function.
- It takes the dot product of data points in the original space.
- A polynomial function is applied to compute relationships in the new space.
- Commonly used in SVM classification problems.
- Effective for datasets that are not linearly separable.
- Helps find a hyperplane that can separate classes more effectively.
- Has tunable parameters, including:
    - Degree of the polynomial, which controls the flexibility of the decision boundary.
    - Coefficient, which affects the scale of the polynomial function.

Polynomial Kernel

- It is a more generalized representation of the linear kernel. It **is not** as preferred as other kernel functions as it is **less efficient** and accurate.
- **Polynomial Kernel Formula**

$$K(x_1, x_2) = (x_1^T x_2 + c)^d$$

  - where **c** is a constant and **x1** and **x2** are vectors in the original space. The parameter **c** can be used to control the **trade-off** between the fit of the training data and the size of the margin. A large **c** value will give a low training error but may result in overfitting. A small **c** value will give a high training error but may result in underfitting.
  - The degree **d** of the polynomial can be used to **control the complexity** of the model. A high degree **d** will result in a more complex model that may overfit the data, while a low degree **d** will result in a simpler model that may underfit the data.

Applications of the Polynomial Kernel

- **Classification Tasks**: Used in text classification (like email filtering) and image recognition (like facial recognition).
- **Regression Problems**: Helps model non-linear relationships in predicting continuous outcomes.
- **Bioinformatics**: Classifies gene expressions related to diseases.
- **Financial Predictions**: Used in stock price forecasting based on historical data.
- **Pattern Recognition**: Effective in recognizing handwritten digits

Consider data points x=(1,2) y=(2,3)

Linear Kernel $k(x,y) = x \cdot y$

Polynomial Kernel $k(x,y) = (x \cdot y + c)^{d}$

Gaussian Radial Kernel

It is one of the most preferred and used kernel functions in svm. It is usually chosen for non-linear data. It helps to make proper separation when there is no prior knowledge of data.

Gaussian Radial Basis Formula

$$K(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)$$

- $\|x - y\|^2$: squared Euclidean distance between x and y.
- σ: parameter that controls the spread (width) of the Gaussian.
- Sometimes, $\gamma = \frac{1}{2\sigma^2}$ is used:

$$K(x, y) = \exp(-\gamma\|x - y\|^2)$$

- If x and y are **very close**, $\|x - y\|^2 \approx 0 \rightarrow K(x, y) \approx 1$.
- If x and y are **far apart**, $\|x - y\|^2$ is large $\rightarrow K(x, y) \approx 0$.
- So, the kernel gives a **similarity score between 0 and 1**.

Gaussian Radial Kernel

- Unlike the **linear kernel** the Gaussian kernel can **map data into infinite-dimensional space**, making it possible to separate data that is not linearly separable in the original space.
- It's flexible and powerful because it adapts to the structure of the data.

- The RBF kernel helps us understand how close or similar two points (like pieces of data) are in a way that can help with making predictions.
- **How It Works**:
  - It looks at the distance between the two points. If they are close together, they get a high similarity score (close to 1). If they are far apart, the score gets lower (closer to 0).
  - This is done using a mathematical formula that includes an exponential function, which means it uses a smooth curve to determine similarity.
- **Key Parameter (σ)**:
  - The kernel uses a value called σ to control how much distance matters:
    - A small σ means only very close points will be considered similar.
    - A large σ means that even points that are farther apart can still be seen as somewhat similar.

Applications of the RBF Kernel

- **Classification Tasks**: The RBF kernel is commonly used in SVM classification problems, especially when the data is not linearly separable. It helps create decision boundaries that can effectively separate different classes.
- **Regression Problems**: The RBF kernel can also be used in regression tasks, allowing models to fit complex patterns in the data.
- **Image Recognition**: In computer vision, the RBF kernel helps classify images based on features extracted from them.
- **Natural Language Processing**: It can be used to classify text data by measuring the similarity between different text samples.

Support Vector Machine Use Cases

- **Image Classification**: Identifying objects in images, like recognizing faces or animals.
- **Text Categorization**: Sorting documents into categories like spam or not-spam in emails.
- **Bioinformatics**: Classifying proteins or genes based on biological data.
- **Handwriting Recognition**: Identifying letters and digits in handwriting.
- **Stock Market Prediction**: Predicting stock price trends based on historical data

Advantages of SVM:

- **Works well with lots of features**: SVM is good at handling data with many characteristics or dimensions.
- **Handles small datasets with many features**: It can still perform well even if there are more features than data points.
- **Saves memory**: SVM only uses some of the training data (called support vectors) to make predictions, so it doesn't need a lot of memory.
- **Flexible**: You can choose different methods (called kernels) to separate data. There are standard ones available, but you can also create your own.
- **Good at avoiding over fitting**: SVM tries to make the separation between classes as wide as possible, helping it perform better on new, unseen data.
- **Handles both simple and complex data**: SVM can work with both straight-line separations (linear) and more complex curves (non-linear) using the right kernel.

Disadvantages of SVM:

- **Not good for big datasets**: SVM can be slow and take a lot of time to process when there's too much data.
- **Sensitive to messy data**: If there's noisy or incorrect data, SVM might not work as well.
- **Hard to pick the right settings**: Choosing the best method (kernel) and parameters for SVM can be tricky and may take several tries.
- **Training the SVMs is inefficient and time consuming process when data is large.**
- **Works only for binary classification:** one have to make cascade of SVMs to get around this constraint
- **Not great for giving probabilities**: SVM doesn't naturally provide probabilities for its predictions, ie confidence level for classifying an instance.